

Open and Reproducible Science in R

Technical Handbook

Ernest Guevarra Proochista Ariana

18 November 2025

Table of contents

Preface	1
Organisation	1
1. Tools	1
2. Practices	2
3. Processes	2
How to use	2
Introduction	3
All about R	3
Open and reproducible science	4
I. Tools	7
1. Installing and setting up tools	9
1.1. Installing R, RStudio, and git	9
1.2. Windows	9
Step 1: Download and install R	9
Step 2: Download and install RStudio	10
Step 3: Download and install Rtools	10
Step 4: Download and install Git for Windows	10
1.3. macOS	11
Step 1: Download and install R	11
1.3.1. Install RStudio	11
1.3.2. Install git for macOS	11

Table of contents

1.4. Register a GitHub account	12
Step 1. Sign-up to GitHub	12
Step 2. Set a GitHub username	14
Step 3. Setup two-factor authentication (2FA)	15
Step 4. Get added to the Oxford IHTM GitHub organisation	16
2. Introduction to R and RStudio	17
2.1. What is R?	17
2.2. Why use R?	18
2.3. What is RStudio	19
3. Introduction to git and GitHub	21
3.1. All about git	21
3.2. All about GitHub	21
4. Connecting RStudio with GitHub	23
4.1. Introduce yourself to git	23
4.2. Create a GitHub personal access token (PAT)	25
Step 1: Go to Settings from your GitHub account menu . .	26
Step 2: From Settings navigate to Developer Settings . . .	26
Step 3: From Developer Settings navigate to Personal Access Token	27
Step 4: Select Tokens (classic)	28
Step 5: Click on Generate new token	29
Step 6: Select Generate new token (classic)	30
Step 7: Give your token a name	31
Step 8: Set an expiry date for the token	32
Step 9: Set scopes	33
Step 10: Click on Generate token	34
Step 11: Store your PAT	34

II. Practices	37
5. Getting acquainted with R	39
5.1. Summary	91
6. Writing functions	93
 III. Processes	 107
7. Cloning a GitHub repository into your local computer using RStudio	109
7.1. Get the GitHub repository URL	110
1. Go to the repository's GitHub page	111
2. Copy the repository URL	111
7.2. Go to RStudio and create new project	112
7.3. Choose Version Control	113
7.4. Select Git	114
7.5. Setup repository settings	115
1. Paste the repository URL you copied earlier	116
2. Set the project directory name	116
3. Set local directory	116
4. Create project	116
 8. Committing your changes and pushing them to GitHub	 117
8.1. Click on Commit in the Git tab on RStudio	117
8.2. Getting changes saved and push to GitHub	118
1. Stage changes	119
2. Add a commit message	119
3. Click on the Commit button	120
4. Click on the Push button	120
8.3. Initiate a pull request	120
1. Click on the branches link from your repository	120
2. Make a pull request	120

Table of contents

3. Enter a title for your pull request	121
4. Create a pull request	122
5. Wait for review	123
9. Participating in an existing R/RStudio project	125
9.1. Clone the project to your local machine	125
9.2. Create a new branch from the main branch	125
9.2.1. Click on New Branch	126
9.2.2. Name the new branch	127
9.3. Code and make changes to your branch	128
9.4. Commit and push your changes and initiate a pull request .	129
9.5. Merge pull request	129
10. Initiating an R/RStudio project	131
10.1. Create a new project in RStudio	132
10.1.1. Click on New Project button on RStudio	132
10.1.2. Create a New Directory	134
10.1.3. Select New Project as project type	135
10.1.4. Specify details for new project	136
10.2. 2. Structure/organise your new project appropriately	140
10.3. 3. Start coding	140
10.4. Next steps	141
11. Creating portable and reproducible scientific workflows	143
11.1. Create a new RStudio project	145
11.2. Create an R file for package dependencies	147
11.3. Create placeholder directories	148
11.4. Create the target script file	149
11.5. Edit the targets script file	150
12. Contributing to Oxford iHealth projects	153
12.1. Research software development	153
12.1.1. Get familiar with R's package writing process	154

Table of contents

12.1.2. Get familiar and reach intermediate level git and GitHub skills	154
12.1.3. Review our portfolio of research software	155
12.1.4. Communicate with developers	155
12.1.5. Clone or fork the project repository	156
References	157

Preface

The Open and Reproducible Science in R sub-module of the MSc in International Health and Tropical Medicine is designed to equip students with the knowledge and skills necessary to conduct both **academic research** and more importantly **real-world data analysis** that is transparent, reproducible, and in line with the principles of open science.

This technical handbook serves as the *go-to* guide for MSc IHTM students to the various tools, technologies, and processes that they will be learning and using within the module.

Organisation

This handbook is divided into three sections:

1. Tools

This section cover topics on the various tools and technologies that are to be used and/or introduced in the module. The R *language and environment for statistical computing and graphics* is primary of these as the module is specific to R. All other tools and technologies are either built specific for use with R (e.g. RStudio which is the IDE of choice for the module and for this handbook) or are general tools that enhance the *userR* experience and/or supports known and accepted best practices for open and reproducible science using R.

Preface

2. Practices

This section covers topics on recommended best practices for optimal usage and maintenance of R and RStudio.

3. Processes

This section covers topics on scientific/data analysis workflows with a focus on steps in initiating and setting up and participating and contributing to such projects within an open and reproducible framework.

How to use

Even though all efforts have been made to order the chapters in a way that is coherent and logical, this handbook is designed such that chapters are standalone topics in of themselves and uses cross-referencing between chapters to make links to the various learning topics/concepts. As such, the best use of this handbook is to use each chapter as a reference for more in-depth discussion of a topic discussed in class rather than a book to read from start to finish.

Introduction

All about R

R is a language and environment for statistical computing and graphics. It is a GNU project which is similar to the S language and environment which was developed at Bell Laboratories (formerly AT&T, now Lucent Technologies) by John Chambers and colleagues. R can be considered as a different implementation of S. There are some important differences, but much code written for S runs unaltered under R.

R provides a wide variety of statistical (linear and nonlinear modelling, classical statistical tests, time-series analysis, classification, clustering, etc.) and graphical techniques, and is highly extensible. The S language is often the vehicle of choice for research in statistical methodology, and R provides an open source route to participation in that activity.

One of R's strengths is the ease with which well-designed publication-quality plots can be produced, including mathematical symbols and formulae where needed. Great care has been taken over the defaults for the minor design choices in graphics, but the user retains full control.

R is available as free software under the terms of the Free Software Foundation's GNU General Public License in source code form. It compiles and runs on a wide variety of UNIX platforms and similar systems (including FreeBSD and Linux), Windows and MacOS.

R is unique in that it is not general-purpose. It does not compromise by trying to do a lot of things. It does a few things very well, mainly

Introduction

statistical analysis and data visualization. While you can find data analysis and machine learning libraries for languages like Python, R has many statistical functionalities built into its core. No third-party libraries are needed for much of the core data analysis you can do with the language.

But even with this specific use case, it is used in every industry you can think of because a modern business runs on data. Using past data, data scientists and data analysts can determine the health of a business and give business leaders actionable insights into the future of their company.

Just because R is specifically used for statistical analysis and data visualization doesn't mean its use is limited. It's actually quite popular, ranking 19th in the TIOBE index of the most popular programming languages.

Academics, scientists, and researchers use R to analyze the results of experiments. In addition, businesses of all sizes and in every industry use it to extract insights from the increasing amount of daily data they generate.

Open and reproducible science

Open and reproducible science is the practice of science in such a way that others can collaborate and contribute and where research data, lab notes and other research processes are freely available, under terms that enable reuse, redistribution and reproduction of the research and its underlying data and methods. Reproducible research means that research data and code are made available so that others are able to reach the same results as are claimed in scientific outputs. Closely related is the concept of replicability, the act of repeating a scientific methodology to reach similar conclusions. These concepts are core elements of empirical research.

Open science is important because it enhances the **accessibility**, **transparency**, and **collaboration** of scientific research.

Open and reproducible science

Open science makes research data, publications, and resources freely available to anyone, regardless of their location, institutional affiliation, or financial situation. This democratizes knowledge and ensures that even those outside of well-funded research institutions can access the latest scientific findings.

By making data, methods, and results openly available, open science allows other researchers to verify, replicate, and build upon previous work. This transparency is essential for the self-correcting nature of science, helping to ensure the reliability and integrity of research findings.

When data and findings are openly shared, other researchers can more quickly build on existing work, leading to faster scientific progress. This is particularly important in fields like medicine or environmental science, where rapid advancements can have significant societal impacts.

Open science fosters collaboration across disciplines, institutions, and borders. Researchers can combine their expertise and resources to tackle complex problems, leading to more innovative solutions. Open data and resources also encourage citizen science, where the general public can contribute to scientific research.

By making research processes and findings open and accessible, science becomes more transparent to the public, which can increase trust in scientific research. Open science also allows the public to engage more directly with science, fostering a greater understanding and appreciation of scientific work.

Open science reduces duplication of effort by making data and methods available for reuse. Researchers can build on existing work rather than starting from scratch, which can save time and resources. Additionally, open access to research outputs can reduce costs for institutions and researchers who would otherwise need to pay for access to publications.

Many of the world's most pressing challenges, such as climate change, pandemics, and poverty, require global collaboration and knowledge-sharing. Open science facilitates this by making research outputs accessible to

Introduction

scientists and policymakers worldwide, particularly in low- and middle-income countries that may lack access to expensive scientific resources.

In essence, open science enhances the efficiency, equity, and impact of scientific research, making it a critical approach for advancing knowledge and addressing global challenges.

The **Open and Reproducible Science in R** module is designed to give MSc IHTM students a foundational understanding and appreciation of the pillars of open science more broadly and within that the concepts, methods and tools for reproducible research more specifically. To further the students' learning, practical examples and exercises are walked through and discussed using the R language for statistical computing as a way to practically demonstrate these concepts.

Part I.

Tools

1. Installing and setting up tools

Following are the steps to installing R, RStudio, and Git depending on your operating system.

1.1. Installing R, RStudio, and git

1.2. Windows

Step 1: Download and install R

Important that R is installed first. R is the main software and is needed for RStudio to work properly. R should always be installed first.

Go to <https://cran.r-project.org> and click on the link that says ***Download R for Windows***. In the following page, click on the link that says ***install R for the first time***.

Then click on ***Download R-4.X.X for Windows*** (latest release version). This will start the download process.

Once downloaded, go to the **.exe** file in your **Downloads** folder, double-click and follow all the install prompts, selecting recommended options all the time.

1. Installing and setting up tools

Step 2: Download and install RStudio

This step requires that **Step 1** has been done and was successful.

Go to <https://posit.co/download/rstudio-desktop/> and select the download specific for your Windows machine.

Once downloaded, double-click on **.exe** file downloaded to your **Downloads** folder and then follow all install prompts, always selecting recommended options.

Step 3: Download and install Rtools

For the things that you will be taught in the **Open and Reproducible Science** sub-module, you will need to expand the installation of R by installing the **Rtools** software.

Go to <https://cran.r-project.org/bin/windows/Rtools/> and choose to download the latest version of the installer (which is the Rtools version compatible with the R version you have installed in Step 1).

Once you have downloaded the **.exe** file, double-click on the **.exe** file and follow all install prompts. Choose all the recommended options.

Step 4: Download and install Git for Windows

For the things that you will be taught in the **Open and Reproducible Science** sub-module, you will need to install **Git for Windows**.

Go to this link - <https://github.com/git-for-windows/git/releases/latest> - to download the latest version of git. Make sure to select the version compatible with your Windows machine (64-bit or 32-bit).

Once you have downloaded the **.exe** file, double-click it and then follow all install prompts. Choose all recommended options.

1.3. macOS

Step 1: Download and install R

Important that R is installed first. R is the main software and is needed for RStudio to work properly. R should always be installed first.

Go to <https://cran.r-project.org> and click on the link that says **Download R for macOS**. In the following page, you will have two choices of R versions to install. Make sure to install the appropriate version for your macOS version (Apple Silicon vs Apple Intel version). Click on the download link for your macOS version. This will start the download process of the `.pkg` file specific for installing in macOS computers.

Once downloaded, go to the `.pkg` file in your **Downloads** folder, double-click and follow all the install prompts, selecting recommended options all the time.

1.3.1. Install RStudio

This step requires that **Step 1** has been done and was successful.

Go to <https://posit.co/download/rstudio-desktop/> and select the download specific for your macOS machine.

Once downloaded, double-click on `.dmg` file downloaded to your **Downloads** folder and then follow all install prompts, always selecting recommended options.

1.3.2. Install git for macOS

For the things that you will be taught in the **Open and Reproducible Science** sub-module, you will need to install **git for macOS**. Apple machines are already pre-installed with `git` but it is usually an Apple

1. Installing and setting up tools

specific version of git and tends to be older and not configured in the way we need it. So we need to install another version of it that comes with Apple's Xcode command line tools.

To install, go to the macOS terminal and type the following command:

```
xcode-select --install
```

1.4. Register a GitHub account

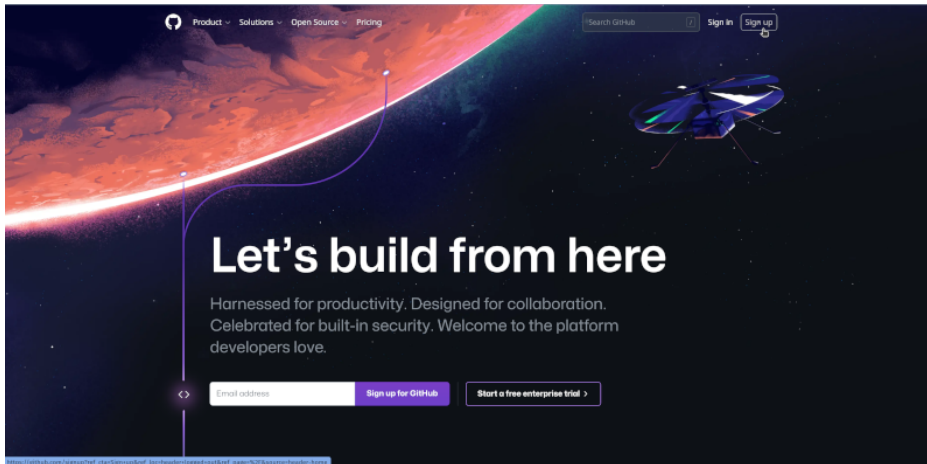
For the **Open and Reproducible Science in R** sub-module, you will need a GitHub account to be able to receive the code materials and assignments that will be provided. This is the mechanism by which these materials will be distributed. Hence you will need to register an account with GitHub (if you don't already have one). It's free!

Step 1. Sign-up to GitHub

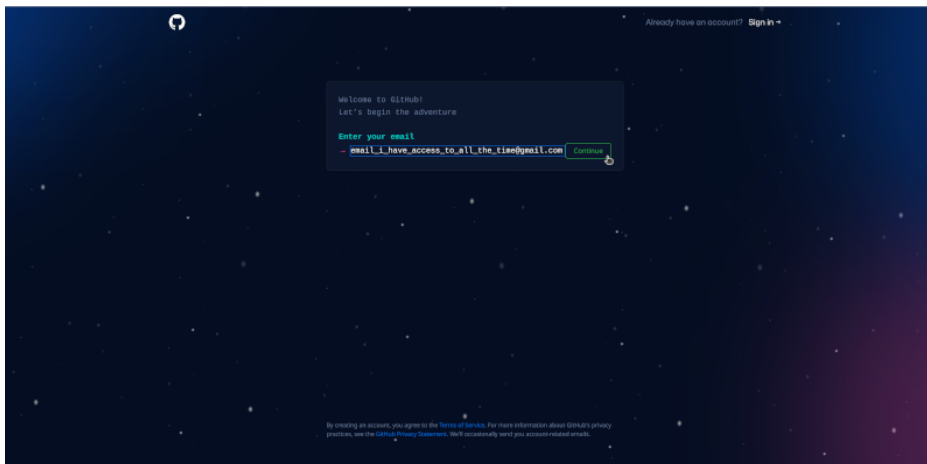
Go to <https://github.com>.

On the upper right hand corner of the page, click on ***Sign-up*** button

1.4. Register a GitHub account



You will be then prompted to provide an email address to register your account with.

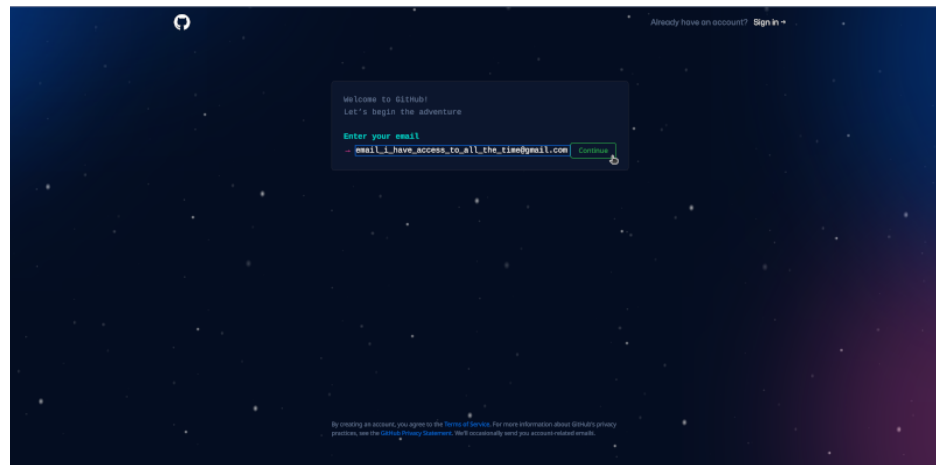


1. Installing and setting up tools

i Note

With regard to the email address to use for creating a GitHub account, best practice is to use an email address that you will have access to all the time. Email addresses such as those for school (if you are a student) or for your current work may not always be the best email address to use as these email addresses tend to be time-limited (i.e., you lose the email address once you graduate or once you leave your current work).

You will then be prompted for a password



Then follow all other prompts after this including confirmation of your email and creating a GitHub username (see next step).

Step 2. Set a GitHub username

With regard to creating/selecting a GitHub username, following are some best practice recommendations (Jenny Bryan and Jim Hester n.d.).

1.4. Register a GitHub account

Tips for selecting GitHub username

- Incorporate your actual name as this lets people know who they're dealing with and also makes your username easier for people to guess or remember.
- Reuse your username from other contexts, e.g., Twitter or Slack.
- Pick a username that will be appropriate revealing to a future boss.
- Shorter is better than longer.
- Be as unique as possible in as few characters as possible.
- Make it timeless and context-agnostic. Don't add a date or year or a reference to your current location, university, or employer.
- Avoid the use of upper vs. lower case to separate words. We highly recommend all lowercase. A better strategy for word separation is to use a hyphen (-).

Step 3. Setup two-factor authentication (2FA)

It is important to keep your GitHub account secure. Any breach in security of your online accounts, including GitHub, not only affects you but also those that you collaborate with. To increase the security of your GitHub account, please enable two-factor authentication (2FA) for your account. This can be done [here](#). There are 4 options for 2FA in GitHub. We recommended enabling at least 2 of these options. If you are familiar with use of passkeys, we recommend using this authentication approach in addition to 2FA.

1. Installing and setting up tools

Step 4. Get added to the Oxford IHTM GitHub organisation

The Oxford iHealth GitHub organisation is the organisational GitHub account for the MSc IHTM. To be included in the organisation, share your GitHub username to the sub-module lead who will then add you to the organisation. This is an important step as assignments and exercises for the sub-module are distributed through GitHub and GitHub Classroom via this organisational account.

! Important

The Oxford iHealth organisational GitHub account requires members to have 2FA activated. It is therefore imperative that you enable 2FA on your account to be included in the organisation.

i Note

You will soon receive a message at the email address you registered to GitHub with inviting you to join the Oxford iHealth organisation. Accept the invitation.

2. Introduction to R and RStudio

2.1. What is R?

R is a system for data manipulation, calculation, and graphics. It provides:

- Facilities for data handling and storage
- A large collection of tools for data analysis
- Graphical facilities for data analysis and display
- A simple but powerful programming language

R is often described as an environment for working with data. This is in contrast to a statistical *package* which is a collection of very specific tools. R is not strictly a statistics system but a system that provides many classical and modern statistical procedures as part of a broader data-analysis tool. This is an important difference between R and other statistical systems. In R a statistical analysis is usually performed as a series of steps with intermediate results being stored in objects. Systems such as SPSS and SAS provide copious output from (e.g.) a regression analysis whereas R will give minimal output and store the results of a fit for subsequent interrogation or use with other R functions. This means that R can be tailored to produce exactly the analysis and results that you want rather than produce an analysis designed to fit all situations.

R is a language based product. This means that you interact with R by typing commands such as:

2. Introduction to R and RStudio

```
table(SEX, LIFE)
```

rather than by using menus, dialog boxes, selection lists, and buttons. This may seem to be a drawback but it means that the system is considerably more flexible than one that relies on menus, buttons, and boxes. It also means that every stage of your data management and analysis can be recorded and edited and re-run at a later date. It also provides an audit trail for quality control purposes.

R is available under UNIX (including Linux), the Apple operating system macOS, and Microsoft Windows. The method used for starting R will vary from system to system. On UNIX systems you may need to issue the R command in a terminal session or click on an icon or menu option if your system has a windowing system. On Apple systems R will be available as an application but can also be run in a terminal session. On Microsoft Windows systems there will usually be an icon on the Start menu or the desktop.

2.2. Why use R?

R is an open source system and is available under the *GNU general public license* (GPL) which means that it is available for free but that there are some restrictions on how you are allowed to distribute the system and how you may charge for bespoke data analysis solutions written using the R system. Details of the general public license are available from <http://www.gnu.org/copyleft/gpl.html>.

R is available for download from <http://www.r-project.org/>.

This is also the best place to get extension packages and documentation. You may also subscribe to the R mailing lists from this site. R is supported through mailing lists. The level of support is at least as good as for commercial packages. It is typical to have queries answered in a matter of a few hours.

2.3. *What is RStudio*

Even though R is a free package it is more powerful than most commercial packages. Many of the modern procedures found in commercial packages were first developed and tested using R or S-Plus (the commercial equivalent of R).

2.3. What is RStudio

RStudio is an **integrated development environment (IDE)** for R. It includes a console, syntax-highlighting editor that supports direct code execution, as well as tools for plotting, history, debugging and workspace management.

RStudio is available in open source and commercial editions and runs on the desktop (Windows, macOS, and Linux) or in a browser connected to RStudio Server or RStudio Workbench (Debian/Ubuntu, Red Hat/CentOS, and SUSE Linux).

<https://youtu.be/n3uue28FD0w?si=DAMZrT6xhLS8ZMLH>

3. Introduction to git and GitHub

3.1. All about git

git is a version control system for software development. It allows developers to keep track of changes made to their code and collaborate with other developers on a project. git also allows for easy rollbacks and branch management. It is widely used in the software industry and is considered one of the best version control systems available.

git was developed by Linus Torvalds in 2005. He created git as a replacement for the proprietary version control system he was using at the time. The development of git was driven by the need for a distributed version control system, which allows multiple developers to work on a project simultaneously, without the need for a central server. Linus Torvalds is also known for creating the Linux operating system kernel.

To use git in your machines, you will need to install it as described in Section 1.1.

3.2. All about GitHub

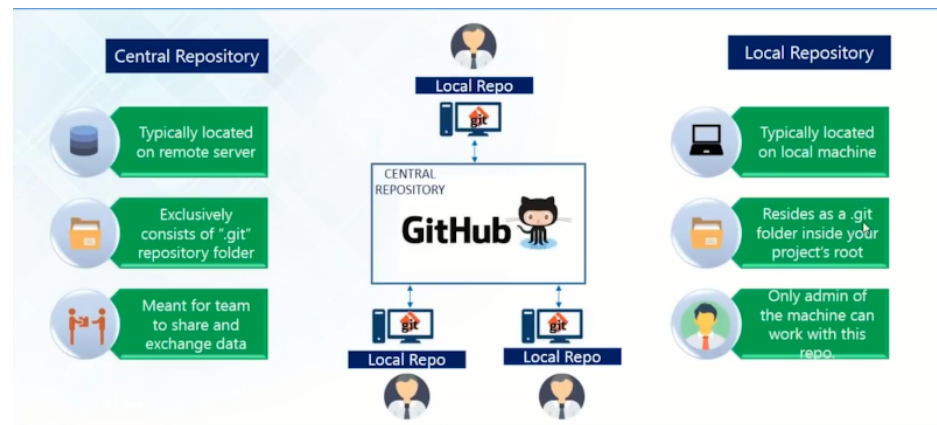
GitHub is a web-based platform that provides hosting for software development and a community of developers to collaborate, share and learn from each other. It is built on top of git, which is the version control system used for managing and tracking changes to the code. Developers can use GitHub to store and manage their code, collaborate with other developers,

3. Introduction to git and GitHub

and track and manage issues and bugs. It also provides tools for code review, project management, and documentation. It is widely used by developers and organizations to host and share code, as well as to build and maintain open-source software.

GitHub is not a software you need to install. Rather it is a remote or cloud-based server that holds its users' code versioned using the git version control system and to which a user's local, git-versioned code syncs/communicates with.

A good illustration of the git and GitHub relationship can be viewed below:



4. Connecting RStudio with GitHub

Once RStudio is installed and a GitHub account has been created and registered, the final step in the R, RStudio, git, and GitHub dance is to create git-related settings on your machine that will identify you as a unique/specific git user and then creating a GitHub **personal access token (PAT)** which will serve as the key or proof that you will use to identify yourself as the git user you claim to be whenever you try to syn/connect to your GitHub account via RStudio or any other tool on your local computer.

Follow are the steps you will need to perform:

4.1. Introduce yourself to git

Open RStudio and then select the ***Terminal*** tab in the console pane.

The terminal is the tool used to interface with your computer through commands written in a programming language called **bash**. Most of you would have never used the terminal because we mostly use software built-in our computers that provide a ***graphical user interface (GUI)*** to perform operations and tasks.

RStudio comes with the terminal tool built in and usually is already available from the console pane as a separate table along side the R console (see below). By default, whenever you open RStudio, this tab for the terminal tool should already be available.

4. Connecting RStudio with GitHub

If it is not present, you can easily open a new terminal tab in the console pane by going to the RStudio menu ribbon and clicking on:

Tools --> Terminal --> New Terminal

or you can use the ALT + SHIFT + R keyboard shortcut.

In this step, you are basically going to issue a set of commands to your computer to save and store specific settings for the git software that you have installed.

Specifically, you are going to let git know who you are (your name) and what your email address (associated with your GitHub account) is.

The commands will be issued on the terminal. The commands are:

```
git config --global user.name 'YOUR FULL NAME'
git config --global user.email 'YOUR EMAIL ADDRESS'
```

Make sure to supply your full proper name (and not your username you created in GitHub).

Make sure that the email you provide is the email address you used to register and create an account with GitHub.

Unless there was an error in your syntax, you should not expect any output on the terminal after you issue the commands. To check that your name and email address have been recorded and/or that the name and email address recorded is correct, you can issue the following command:

```
git config --global --list
```

Here is an example of what you will see after issue this command:

```
user.name=YOUR FULL NAME
user.email=YOUR EMAIL ADDRESS
```

4.2. Create a GitHub personal access token (PAT)

Check this output to what you expect it to be specified as. If the information is correct, then you've completed this step. If you need to correct any of this information, then repeat this step making sure that the name and email address you provide is correct.

4.2. Create a GitHub personal access token (PAT)

Now that you have introduced yourself to the git that is installed in your local machine/computer, you should now visit GitHub on a browser and create a *personal access token (PAT)*.

When you communicate/sync/interact with a remote git server, such as GitHub, you have to include credentials in the communication you are sending. These credentials prove that you are a specific GitHub user, who's allowed to do whatever you're asking to do.

git can communicate with a remote server using one of two protocols, HTTPS or SSH, and the different protocols use different credentials.

Here we describe the credential setup for the HTTPS protocol, which is what we recommend if you have no burning reason to pick SSH. With HTTPS, we will use a PAT to connect securely to GitHub from RStudio.

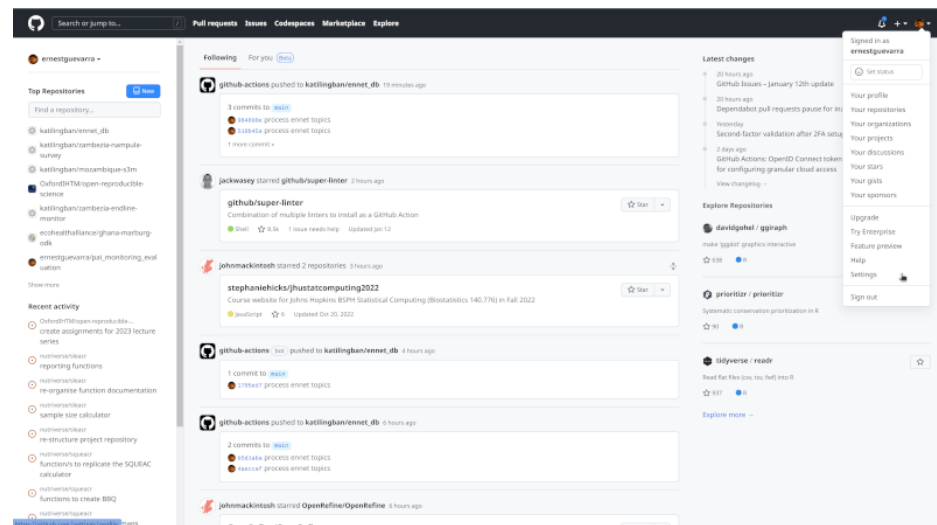
Please note that the PAT is not the same as the password you provided when registering for your GitHub account. Also, in performing a connection to GitHub via HTTPS protocol, your password is not an acceptable credential for communicating with GitHub.

To create a GitHub PAT, you need to:

4. Connecting RStudio with GitHub

Step 1: Go to Settings from your GitHub account menu

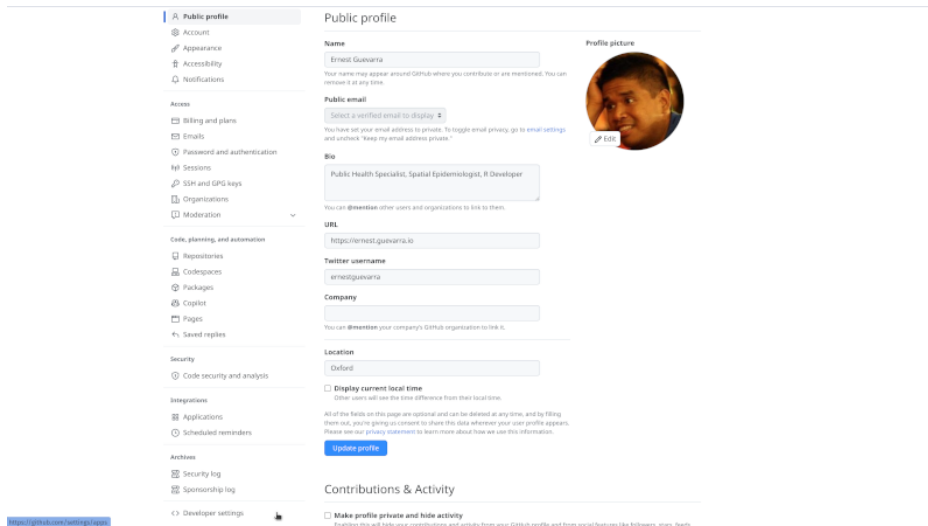
Login to your GitHub account. On the upper right hand corner of the GitHub page you will see your account icon. Click on it to reveal a drop down menu as shown below. Select the *Settings* option.



Step 2: From Settings navigate to Developer Settings

In the *Settings* page, find the *Developer Settings* option on the left hand sidebar as shown below:

4.2. Create a GitHub personal access token (PAT)

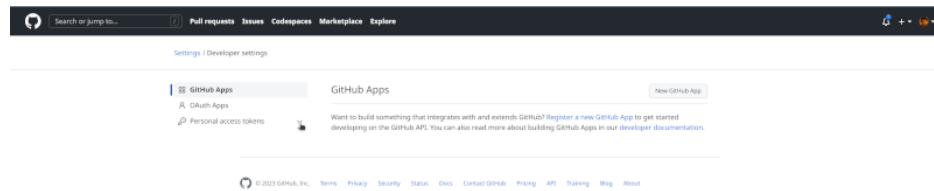


The screenshot shows the GitHub 'Public profile' settings page. On the left is a sidebar with navigation links: Public profile (selected), Account, Appearance, Accessibility, Notifications, Access (Billing and plans, Emails, Password and authentication, Sessions, SSH and GPG keys, Organizations, Moderation), Code, planning, and automation (Repositories, Codespaces, Packages, Copilot, Pages, Saved replies), Security (Code security and analysis), Integrations (Applications, Scheduled reminders), and Actions (Security log, Sponsoring log). The main content area is titled 'Public profile' and includes a 'Profile picture' section with a circular profile picture of Ernest Guervara. Below this are fields for Name (Ernest Guervara), Public email (Select a verified email to display), Bio (Public Health Specialist, Spatial Epidemiologist, & Developer), URL (https://ernestguervara.io), Twitter username (ernestguervara), Company, Location (Oxford), and a checkbox for 'Display current local time'. A 'Update profile' button is at the bottom of the profile section. Below the profile section is the 'Contributions & Activity' section, which includes a checkbox for 'Make profile private and hide activity'.

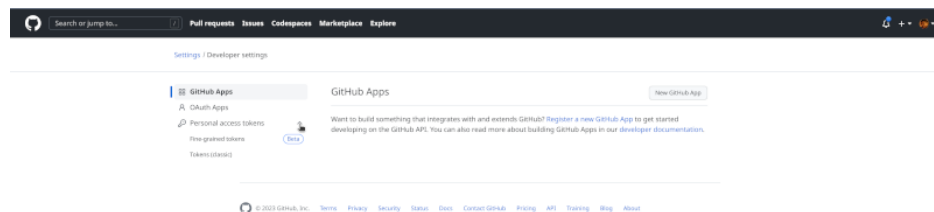
Step 3: From Developer Settings navigate to Personal Access Token

In the *Developer Settings* page, find the *Personal Access Token* option on the left hand sidebar as shown below:

4. Connecting RStudio with GitHub

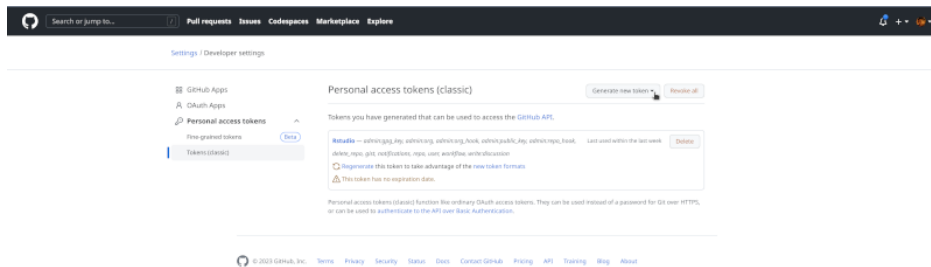


Step 4: Select Tokens (classic)



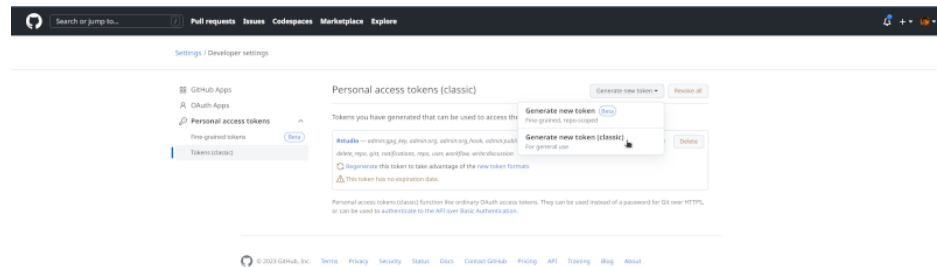
4.2. Create a GitHub personal access token (PAT)

Step 5: Click on Generate new token



4. Connecting RStudio with GitHub

Step 6: Select Generate new token (classic)



4.2. Create a GitHub personal access token (PAT)

Step 7: Give your token a name

Search or jump to... Pull requests Issues Code spaces Marketplace Explore

Settings / Developer settings

GitHub Apps
OAuth Apps
Personal access tokens
Fine-grained tokens
Tokens (classic)

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to authenticate to the API over Basic Authentication.

Note

What's this token for?

Expiration 30 days The token will expire on Sun, Feb 12 2023

Select scopes

Scopes define the access for personal tokens. Read more about OAuth scopes.

☒ repo	Full control of private repositories
☐ repo:status	Access commit status
☐ repo:deployment	Access deployment status
☐ public_repo	Access public repositories
☐ repo:hooks	Access repository invitations
☐ security_events	Read and write security alerts
☐ workflow	Update GitHub Action workflows
☐ writepackages	Upload packages to GitHub Package Registry
☐ readpackages	Download packages from GitHub Package Registry
☐ deletepackages	Delete packages from GitHub Package Registry
☐ admin:org	Full control of orgs and teams, read and write org projects
☐ org:read	Read org and team membership, read and write org projects
☐ org:write	Manage org members and runner groups
☐ admin:public_key	Full control of user public keys
☐ write:public_key	Write user public keys
☐ read:public_key	Read user public keys
☐ admin:repo_hook	Full control of repository hooks

The token name should be short but descriptive of the where or how you will use the token. Since we are using this to connect and allow communication between RStudio and GitHub, **rstudio** can be a good name that would remind you that this is what you are using to secure the connection/communication between your RStudio and your GitHub account.

4. Connecting RStudio with GitHub

Step 8: Set an expiry date for the token

Search or jump to... Pull requests Issues CodeSpaces Marketplace Explore

Settings / Developer settings

GitHub Apps OAuth Apps Personal access tokens Fine-grained tokens (New) Tokens (classic)

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to authenticate to the API over Basic Authentication.

Note

What's this token for?

Expiration ⁺
30 days 8 The token will expire on Sun, Feb 12 2023

Select scopes
Scopes define the access for personal tokens. [Read more about OAuth scopes.](#)

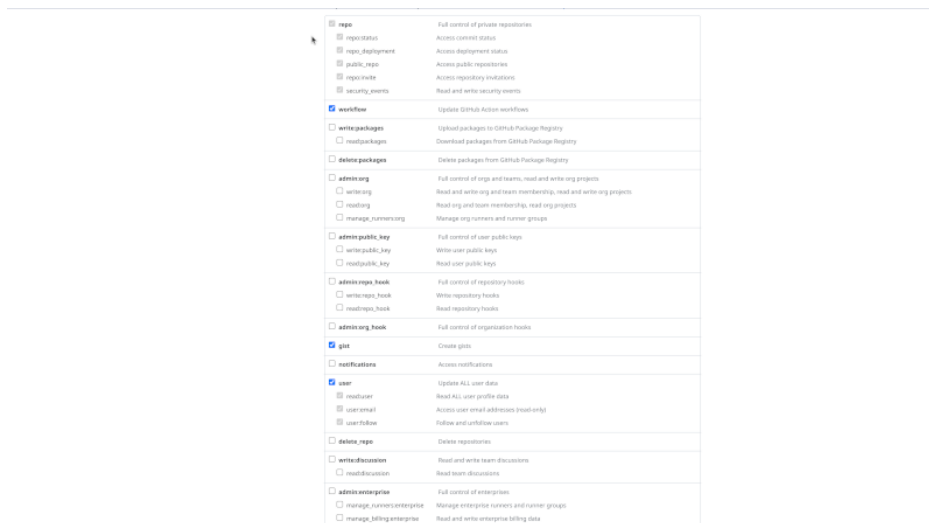
☐ repo	Full control of private repositories
☐ repo:status	Access commit status
☐ repo_deployment	Access deployment status
☐ public_repo	Access public repositories
☐ repo:write	Access repository invitations
☐ security_events	Read and write security events
☐ workflow	Update GitHub Action workflows
☐ write:packages	Upload packages to GitHub Package Registry
☐ read:packages	Download packages from GitHub Package Registry
☐ delete:packages	Delete packages from GitHub Package Registry
☐ admin:org	Full control of orgs and teams, read and write org projects
☐ write:org	Read and write org and team membership, read and write org projects
☐ read:org	Read org and team membership, read org projects
☐ manage:currentorg	Manage org members and runner groups
☐ admin:public_key	Full control of user public keys
☐ write:public_key	Write user public keys
☐ read:public_key	Read user public keys
☐ admin:repo_hooks	Full control of repository hooks

By default, GitHub will set a *30 day validity* for any new token created. Clicking on the option menu will show the other possible time periods to choose from including *No expiry date*.

It is best practice to assign an expiry date for security tokens such as the GitHub PAT. And a 30 day validity is standard practice. However, in reality, it is cumbersome to be creating new tokens frequently and for beginners, having to go through these steps again can be quite a chore. For the purposes of this lecture series, we would recommend setting the expiry for about 90 days to cover the whole period and then as a group, we'll have a *renew GitHub PAT party* on our last session.

4.2. Create a GitHub personal access token (PAT)

Step 9: Set scopes



☒ repo	Full control of private repositories
☐ repo:status	Access repository status
☐ repo:deployment	Access deployment status
☐ public_repo	Access public repositories
☐ repo:write	Access repository instances
☐ security_events	Read and write security events
☒ workflow	Update GitHub Actions workflows
☐ write_packages	Upload packages to GitHub Package Registry
☐ read_packages	Download packages from GitHub Package Registry
☐ delete_packages	Delete packages from GitHub Package Registry
☐ admin_org	Full control of orgs and teams, read and write org projects
☐ write_org	Read and write org and team membership, read and write org projects
☐ read_org	Read org and team membership, read org projects
☐ manage_runners_org	Manage org runners and runner groups
☐ admin_public_key	Full control of user public keys
☐ write_public_key	Write user public keys
☐ read_public_key	Read user public keys
☐ admin_repo_hook	Full control of repository hooks
☐ write_repo_hook	Write repository hooks
☐ read_repo_hook	Read repository hooks
☐ admin_org_hook	Full control of organization hooks
☒ gist	Create gists
☐ notifications	Access notifications
☒ user	Update full user data
☐ read_user	Read full user profile data
☐ user_email	Access user email addresses (read-only)
☐ user_follow	Follow and unfollow users
☐ delete_repo	Delete repositories
☐ write_discussion	Read and write team discussions
☐ read_discussion	Read team discussions
☐ admin_enterprise	Full control of enterprises
☐ manage_runners_enterprise	Manage enterprise runners and runner groups
☐ manage_billing_enterprise	Read and write enterprise billing data

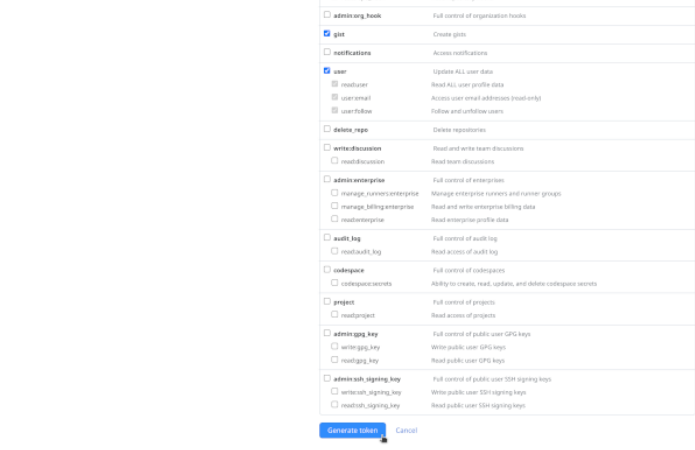
Scopes are the types of permissions that you are attaching the token you are generating. This is again a security feature as tokens should only be given specific and limited permissions based on what you intend the token to be used for. It is not good practice to give a token complete or unlimited permissions as you are exposing your account to high risk if and when your token gets compromised.

For general R users, the following scopes are currently recommended:

- repo
- workflow
- gist
- user

4. Connecting RStudio with GitHub

Step 10: Click on Generate token



Category	Permission
☐ admin:org_hooks	Full control of organization hooks
☒ gist	Create gists
☐ notifications	Access notifications
☒ user	Update ALL user data
☒ read:user	Read ALL user profile data
☒ user:email	Access user email addresses (read-only)
☒ user:follow	Follow and unfollow users
☐ delete_repo	Delete repositories
☐ write:discussion	Read and write team discussions
☐ read:discussion	Read team discussions
☐ admin:enterprise	Full control of enterprises
☐ manage_runners:enterprise	Manage enterprise runners and runner groups
☐ manage_billing:enterprise	Read and write enterprise billing data
☐ read:enterprise	Read enterprise profile data
☐ audit_log	Full control of audit log
☐ read:audit_log	Read access of audit log
☐ codespace	Full control of codespaces
☐ codespace:secrets	Ability to create, read, update, and delete codespace secrets
☐ project	Full control of projects
☐ read:project	Read access of projects
☐ admin:org_key	Full control of public user GPG keys
☐ write:org_key	Write public user GPG keys
☐ read:org_key	Read public user GPG keys
☐ admin:ssh_signing_key	Full control of public user SSH signing keys
☐ write:ssh_signing_key	Write public user SSH signing keys
☐ read:ssh_signing_key	Read public user SSH signing keys

[Generate token](#) [Cancel](#)

© 2023 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Docs](#) [Contact GitHub](#) [Pricing](#) [API](#) [Training](#) [Blog](#) [About](#)

After clicking you will now see a long string of characters and numbers which is your GitHub PAT. It is important to remember that once you see the generated GitHub PAT, you should copy this right away and store it securely.

Step 11: Store your PAT

Treat your GitHub PAT in the same way you would treat your password for online accounts. The best way to securely store the GitHub PAT is using a password manager (1Password, LastPass, Bitwarden). If you have a macOS computer, you can save your GitHub PAT into your computer's keychain.

4.2. Create a *GitHub* personal access token (PAT)

Warning

Non-secure password/token storage practices that has been done by other students before are:

- Email their password/token to themselves

If you are using free email services such as Gmail, then this is a highly non-secure method. Others use their University of Oxford email address and argue that this is secure compared to using the free email services. Whilst it is true that a university email account is more secure, email communications and email storage is still one of the most vulnerable places to keep something that is meant to be kept secret.

- Paste the token into a Word document and save in personal computer with the filename GITHUB_PAT.docx
- Paste the token into a Word document and save in Dropbox or in Google Drive

Please **AVOID** these methods.

Part II.

Practices

5. Getting acquainted with R

In this chapter we will use R to read a dataset and produce some descriptive statistics, produce some charts, and perform some simple statistical inference. The aim of the exercise is for you to become familiar with R and some basic R functions and objects.

The first thing we will do, after starting R, is issue a command to retrieve an example dataset:

```
fem <- read.table("fem.dat", header = TRUE)
```

This command illustrates some key things about the way R works.

We are instructing R to assign (using the `<-` operator) the output of the `read.table()` function to an object called `fem`.

The `fem` object will contain the data held in the file `fem.dat` as an R data.frame object:

```
class(fem)
```

```
[1] "data.frame"
```

5. Getting acquainted with R

You can inspect the contents of the `fem` data.frame (or any other R object) just by typing its name:

```
fem
```

	ID	AGE	IQ	ANX	DEP	SLP	SEX	LIFE	WT
1	1	39	94	2	2	2	1	1	2.23
2	2	41	89	2	2	2	1	1	1.00
3	3	42	83	3	3	2	1	1	1.82
4	4	30	99	2	2	2	1	1	-1.18
5	5	35	94	2	1	1	1	2	-0.14
6	6	44	90	NA	1	2	2	2	0.41

Note that the `fem` object is built from other objects. These are the named vectors (columns) in the dataset:

```
names(fem)
```

```
[1] "ID"    "AGE"    "IQ"     "ANX"    "DEP"    "SLP"    "SEX"    "LIFE"    "WT"
```

The `[1]` displayed before the column names refers to the numbered position of the first name in the output. These positions are known as indexes and can be used to refer to individual items. For example:

```
names(fem)[1]
```

```
[1] "ID"
```

```
names(fem)[8]
```

```
[1] "LIFE"
```

```
names(fem)[2:4]
```

```
[1] "AGE" "IQ"  "ANX"
```

5. *Getting acquainted with R*

The data consist of 118 records:

```
nrow(fem)
```

```
[1] 118
```

each with nine variables:

```
ncol(fem)
```

```
[1] 9
```

for female psychiatric patients.

The columns in the dataset are:

ID	Patient ID
AGE	Age in years
IQ	IQ score
ANX	Anxiety (1=none, 2=mild, 3=moderate, 4=severe)
DEP	Depression (1=none, 2=mild, 3=moderate or severe)
SLP	Sleeping normally (1=yes, 2=no)
SEX	Lost interest in sex (1=yes, 2=no)
LIFE	Considered suicide (1=yes, 2=no)

ID	Patient ID
WT	Weight change (kg) in previous 6 months

5. Getting acquainted with R

The first ten records of the `fem` data.frame are:

	ID	AGE	IQ	ANX	DEP	SLP	SEX	LIFE	WT
1	1	39	94	2	2	2	1	1	2.23
2	2	41	89	2	2	2	1	1	1.00
3	3	42	83	3	3	2	1	1	1.82
4	4	30	99	2	2	2	1	1	-1.18
5	5	35	94	2	1	1	1	2	-0.14
6	6	44	90	NA	1	2	2	2	0.41
7	7	31	94	2	2	NA	1	1	-0.68
8	8	39	87	3	2	2	1	2	1.59
9	9	35	-99	3	2	2	1	1	-0.55
10	10	33	92	2	2	2	1	1	0.36

You may check this by asking R to display all columns of the first ten records in the `fem` data.frame:

```
fem[1:10, ]
```

	ID	AGE	IQ	ANX	DEP	SLP	SEX	LIFE	WT
1	1	39	94	2	2	2	1	1	2.23
2	2	41	89	2	2	2	1	1	1.00
3	3	42	83	3	3	2	1	1	1.82
4	4	30	99	2	2	2	1	1	-1.18
5	5	35	94	2	1	1	1	2	-0.14
6	6	44	90	NA	1	2	2	2	0.41
7	7	31	94	2	2	NA	1	1	-0.68
8	8	39	87	3	2	2	1	2	1.59

9	9	35	-99	3	2	2	1	1	-0.55
10	10	33	92	2	2	2	1	1	0.36

5. Getting acquainted with R

The space after the comma is optional. You can think of it as a *placeholder* for where you would specify the indexes for columns you wanted to display. For example:

```
fem[1:10,2:4]
```

displays the first ten rows and the second, third and fourth columns of the `fem` data.frame:

	AGE	IQ	ANX
1	39	94	2
2	41	89	2
3	42	83	3
4	30	99	2
5	35	94	2
6	44	90	NA
7	31	94	2
8	39	87	3
9	35	-99	3
10	33	92	2

NA is a special value meaning *not available* or *missing*.

You can access the contents of a single column by name:

```
fem$IQ
```

```

      [1]  94  89  83  99  94  90  94  87 -99  92  92  94  91  86  90 -
99  91  82
      [19]  86  88  97  96  95  87 103 -99  91  87  91  89  92  84  94  92  96  96
      [37]  86  92 102  82  92  90  92  88  98  93  90  91 -
99  92  92  91  91  86
      [55]  95  91  96 100  99  89  89  98  98 103  91  91  94  91  85  92  96  90
      [73]  87  95  95  87  95  88  94 -99 -99  87  92  86  93  92 106  93  95  95
      [91]  92  98  92  88  85  92  84  92  91  86  92  89 -
99  96  97  92  92  98
      [109]  91  91  89  94  90  96  87  86  89 -99

```

```
fem$IQ[1:10]
```

```
[1]  94  89  83  99  94  90  94  87 -99  92
```

5. Getting acquainted with R

The `$` sign is used to separate the name of the data.frame and the name of the column of interest. Note that R is case-sensitive so that `IQ` and `iq` are *not* the same.

You can also access rows, columns, and individual cells by specifying row and column positions. For example, the `IQ` column is the third column in the `fem` data.frame:

```
fem[,3]
```

```
 [1]  94  89  83  99  94  90  94  87 -99  92  92  94  91  86  90 -
99  91  82
 [19]  86  88  97  96  95  87 103 -99  91  87  91  89  92  84  94  92  96  90
 [37]  86  92 102  82  92  90  92  88  98  93  90  91 -
99  92  92  91  91  86
 [55]  95  91  96 100  99  89  89  98  98 103  91  91  94  91  85  92  96  90
 [73]  87  95  95  87  95  88  94 -99 -99  87  92  86  93  92 106  93  95  98
 [91]  92  98  92  88  85  92  84  92  91  86  92  89 -
99  96  97  92  92  98
[109]  91  91  89  94  90  96  87  86  89 -99
```

```
fem[9, ]
```

```
   ID AGE  IQ ANX DEP SLP SEX LIFE    WT
9   9  35 -99   3   2   2   1   1 -0.55
```

```
fem[9,3]
```

```
[1] -99
```

There are missing values in the IQ column which are all coded as **-99**. Before proceeding we must set these to the special NA value:

```
fem$IQ[fem$IQ == -99] <- NA
```

The term inside the square brackets is also an index. This type of index is used to refer to subsets of data held in an object that meet a particular condition. In this case we are instructing R to set the contents of the IQ variable to NA if the contents of the IQ variable is **-99**.

5. Getting acquainted with R

Check that this has worked:

```
fem$IQ
```

```
[1] 94 89 83 99 94 90 94 87 NA 92 92 94 91 86 90 NA 91 83
[19] 86 88 97 96 95 87 103 NA 91 87 91 89 92 84 94 92 96 90
[37] 86 92 102 82 92 90 92 88 98 93 90 91 NA 92 92 91 91 86
[55] 95 91 96 100 99 89 89 98 98 103 91 91 94 91 85 92 96 90
[73] 87 95 95 87 95 88 94 NA NA 87 92 86 93 92 106 93 95 98
[91] 92 98 92 88 85 92 84 92 91 86 92 89 NA 96 97 92 92 98
[109] 91 91 89 94 90 96 87 86 89 NA
```

We can now compare the groups who have and have not considered suicide.
For example:

```
by(fem$IQ, fem$LIFE, summary)
```

Look at the help for the `by()` function:

```
help(by)
```

Note that you may use `?by` as a shortcut for `help(by)`.

The `by()` function applies another function (in this case the `summary()` function) to a column in a data.frame (in this case `fem$IQ`) split by the value of another variable (in this case `fem$LIFE`).

It can be tedious to always have to specify a data.frame each time we want to use a particular variable. We can fix this problem by ‘attaching’ the data.frame:

```
attach(fem)
```

We can now refer to the columns in the `fem` data.frame without having to specify the name of the data.frame. This time we will produce summary statistics for `WT` by `LIFE`:

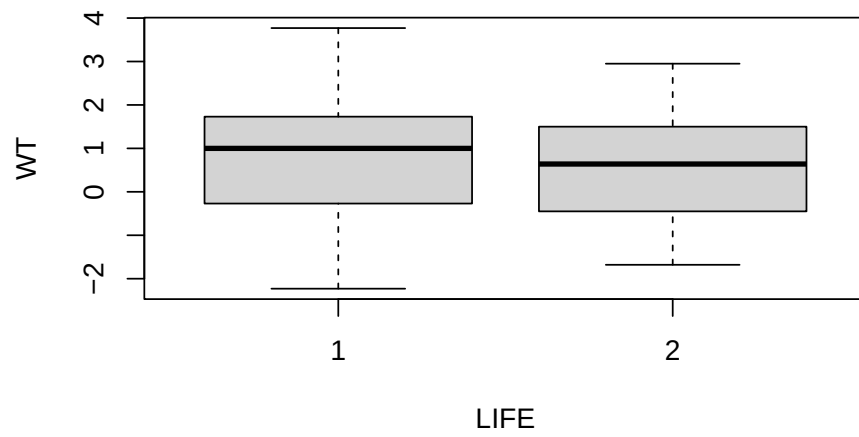
```
by(WT, LIFE, summary)
```

```
LIFE: 1
      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
-2.2300 -0.2700   1.0000   0.7867  1.7300   3.7700      4
-----
LIFE: 2
      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
-1.6800 -0.4500   0.6400   0.6404  1.5000   2.9500      7
```

We can view the same data as a box and whisker plot:

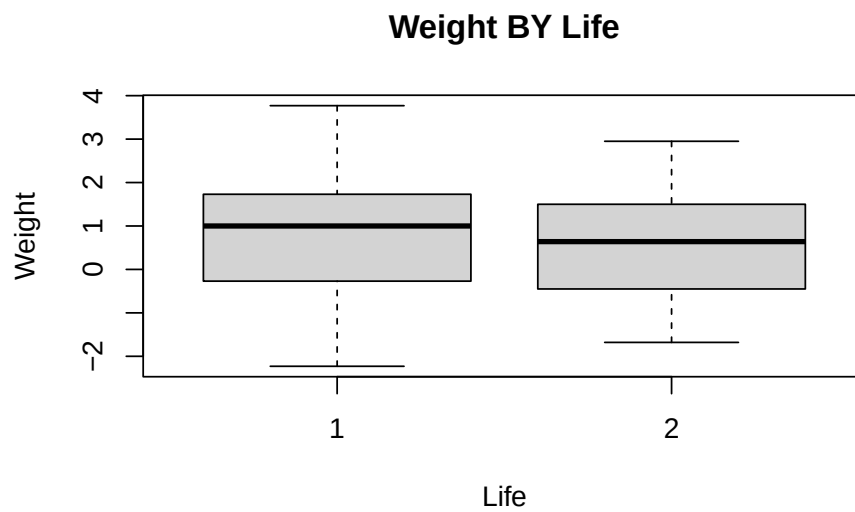
```
boxplot(WT ~ LIFE)
```

5. *Getting acquainted with R*



We can add axis labels and a title to the graph:

```
boxplot(  
  WT ~ LIFE,  
  xlab = "Life",  
  ylab = "Weight",  
  main = "Weight BY Life"  
)
```



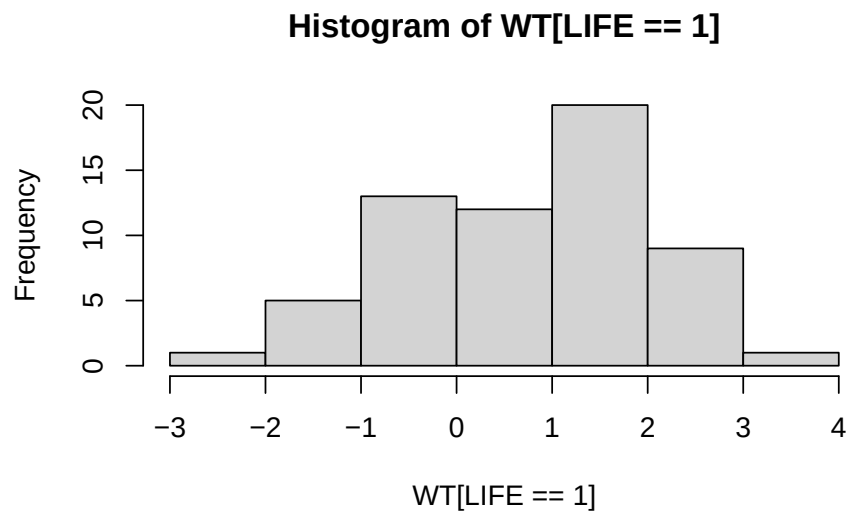
A more descriptive title might be “Weight Change BY Considered Suicide”.

5. *Getting acquainted with R*

The groups do not seem to differ much in their medians and the distributions appear to be reasonably symmetrical about their medians with a similar spread of values.

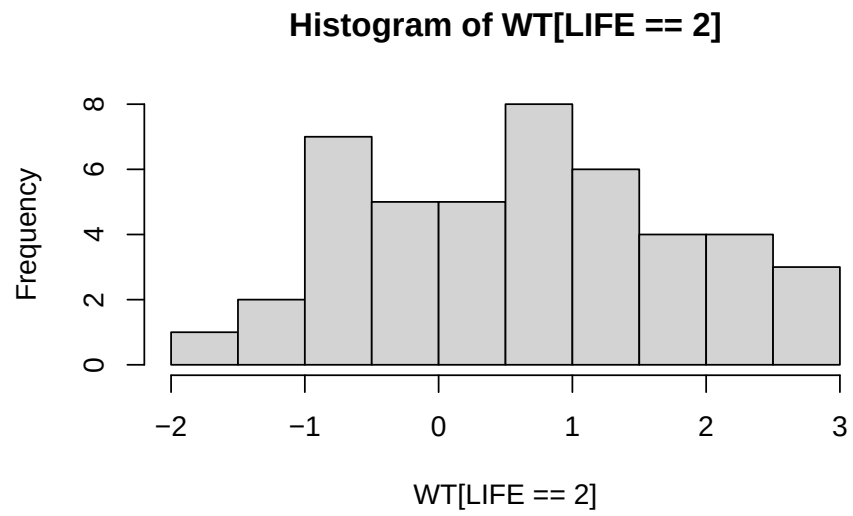
We can look at the distribution as histograms:

```
hist(WT[LIFE == 1])
```



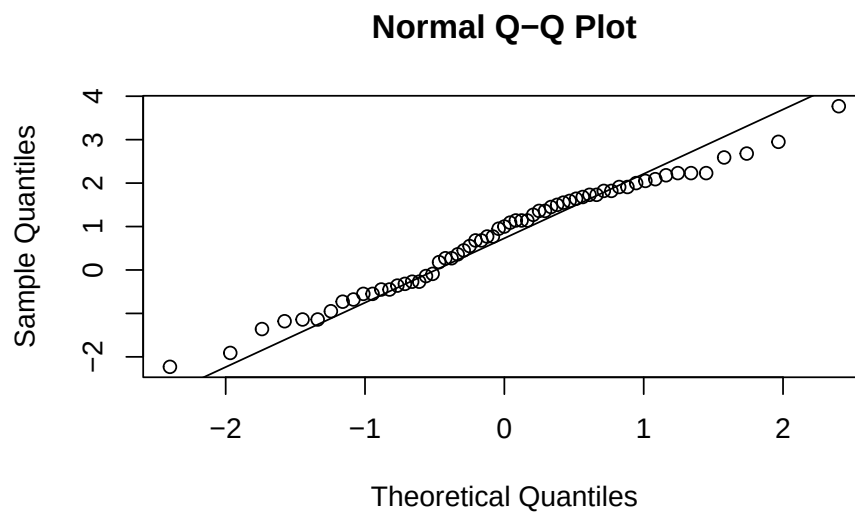
```
hist(WT[LIFE == 2])
```

5. *Getting acquainted with R*



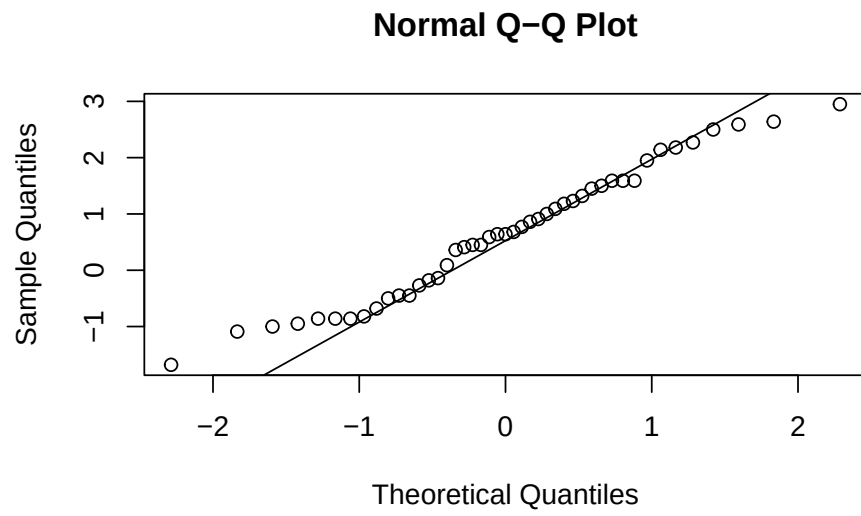
and check the assumption of normality using quantile-quantile plots:

```
qqnorm(WT[LIFE == 1])  
qqline(WT[LIFE == 1])
```



```
qqnorm(WT[LIFE == 2])  
qqline(WT[LIFE == 2])
```

5. *Getting acquainted with R*



or by using a formal test:

```
shapiro.test(WT[LIFE == 1])
```

Shapiro-Wilk normality test

```
data:  WT[LIFE == 1]  
W = 0.98038, p-value = 0.4336
```

```
shapiro.test(WT[LIFE == 2])
```

Shapiro-Wilk normality test

```
data:  WT[LIFE == 2]  
W = 0.97155, p-value = 0.3292
```

Remember that we can use the `by()` function to apply a function to a `data.frame`, including statistical functions such as `shapiro.test()`:

```
by(WT, LIFE, shapiro.test)
```

```
LIFE: 1
```

Shapiro-Wilk normality test

```
data:  dd[x, ]
```

5. *Getting acquainted with R*

W = 0.98038, p-value = 0.4336

LIFE: 2

Shapiro-Wilk normality test

data: dd[x,]

W = 0.97155, p-value = 0.3292

We can also test whether the variances differ significantly using *Bartlett's test* for the homogeneity of variances:

```
bartlett.test(WT, LIFE)
```

```
Bartlett test of homogeneity of variances
```

```
data: WT and LIFE
```

```
Bartlett's K-squared = 0.32408, df = 1, p-value = 0.5692
```

There is no significant difference between the two variances.

Many functions in R have a *formula interface* that may be used to specify multiple variables and the relations between multiple variables. We could have used the formula interface with the `bartlett.test()` function:

```
bartlett.test(WT ~ LIFE)
```

```
Bartlett test of homogeneity of variances
```

```
data: WT by LIFE
```

```
Bartlett's K-squared = 0.32408, df = 1, p-value = 0.5692
```

Having checked the normality and homogeneity of variance assumptions we can proceed to carry out a **t-test**:

5. Getting acquainted with R

```
t.test(WT ~ LIFE, var.equal = TRUE)
```

Two Sample t-test

```
data:  WT by LIFE
t = 0.59869, df = 104, p-value = 0.5507
alternative hypothesis: true difference in means between group 1 and group 2
95 percent confidence interval:
 -0.3382365  0.6307902
sample estimates:
mean in group 1 mean in group 2
      0.7867213      0.6404444
```

There is no evidence that the two groups differ in weight change in the previous six months.

We could still have performed a **t-test** if the variances were not homogeneous by setting the **var.equal** parameter of the **t.test()** function to **FALSE**:

```
t.test(WT ~ LIFE, var.equal = FALSE)
```

Welch Two Sample t-test

```
data:  WT by LIFE
t = 0.60608, df = 98.866, p-value = 0.5459
alternative hypothesis: true difference in means between group 1 and group 2
95 percent confidence interval:
```

```
-0.3326225  0.6251763
sample estimates:
mean in group 1 mean in group 2
      0.7867213      0.6404444
```

or performed a non-parametric test:

```
wilcox.test(WT ~ LIFE)
```

Wilcoxon rank sum test with continuity correction

```
data:  WT by LIFE
W = 1488, p-value = 0.4622
alternative hypothesis: true location shift is not equal to 0
```

5. *Getting acquainted with R*

An alternative, and more general, non-parametric test is:

```
kruskal.test(WT ~ LIFE)
```

Kruskal-Wallis rank sum test

data: WT by LIFE

Kruskal-Wallis chi-squared = 0.54521, df = 1, p-value = 0.4603

We can use the `table()` function to examine the differences in depression between the two groups:

```
table(DEP, LIFE)
```

	LIFE	
DEP	1	2
1	0	26
2	42	24
3	16	1

The two distributions look very different from each other. We can test this using a chi-square test on the table:

```
chisq.test(table(DEP, LIFE))
```

Pearson's Chi-squared test

data: table(DEP, LIFE)

X-squared = 43.876, df = 2, p-value = 2.968e-10

5. Getting acquainted with R

Note that we passed the output of the `table()` function directly to the `chisq.test()` function. We could have saved the table as an object first and then passed the object to the `chisq.test()` function:

```
tab <- table(DEP, LIFE)
chisq.test(tab)
```

Pearson's Chi-squared test

```
data:  tab
X-squared = 43.876, df = 2, p-value = 2.968e-10
```

The `tab` object contains the output of the `table()` function:

```
class(tab)
```

```
[1] "table"
```

```
tab
```

```
      LIFE
DEP   1   2
  1    0 26
  2  42 24
  3  16  1
```

We can pass this table object to another function. For example:

```
fisher.test(tab)
```

Fisher's Exact Test for Count Data

```
data: tab
p-value = 1.316e-12
alternative hypothesis: two.sided
```

When we are finished with the `tab` object we can delete it using the `rm()` function:

```
rm(tab)
```

You can see a list of available objects using the `ls()` function:

```
ls()
```

```
[1] "fem"          "pandoc_dir"    "quarto_bin_path"
```

5. *Getting acquainted with R*

This should just show the `fem` object.

We can examine the association between loss of interest in sex and considering suicide in the same way:

```
tab <- table(SEX, LIFE)
```

```
tab
```

```
      LIFE
SEX  1   2
  1 58 38
  2  5 12
```

```
fisher.test(tab)
```

Fisher's Exact Test for Count Data

```
data:  tab
p-value = 0.03175
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 1.080298 14.214482
sample estimates:
odds ratio
 3.620646
```

Note that with a two-by-two table the `fisher.test()` function produces an estimate of, and confidence intervals for, the odds ratio. Again, we will delete the `tab` object:

```
rm(tab)
```

We could have performed the Fisher exact test without creating the `tab` object by passing the output of the `table()` function directly to the `fisher.test()` function:

```
fisher.test(table(SEX, LIFE))
```

Fisher's Exact Test for Count Data

```
data:  table(SEX, LIFE)
p-value = 0.03175
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 1.080298 14.214482
sample estimates:
odds ratio
 3.620646
```

Choose whichever method you find easiest but remember that it is easy to save the results of any function for later use.

We can explore the correlation between two variables using the `cor()` function:

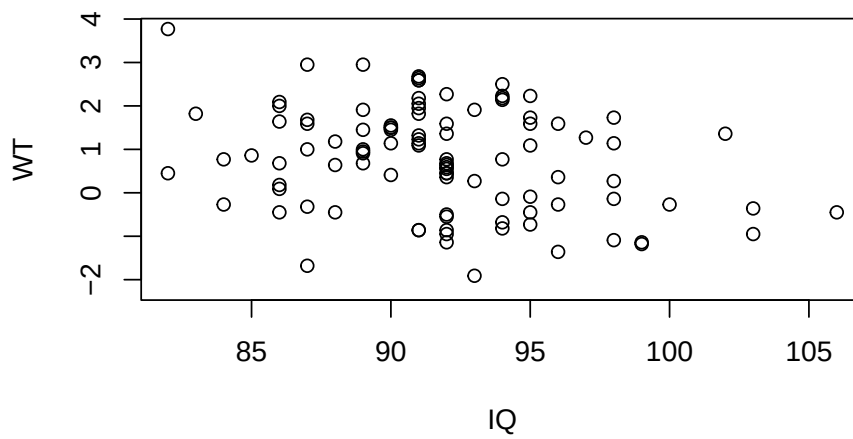
5. *Getting acquainted with R*

```
cor(IQ, WT, use = "pairwise.complete.obs")
```

```
[1] -0.2917158
```

or by using a scatter plot:

```
plot(IQ, WT)
```



and by a formal test:

```
cor.test(IQ, WT)
```

Pearson's product-moment correlation

5. *Getting acquainted with R*

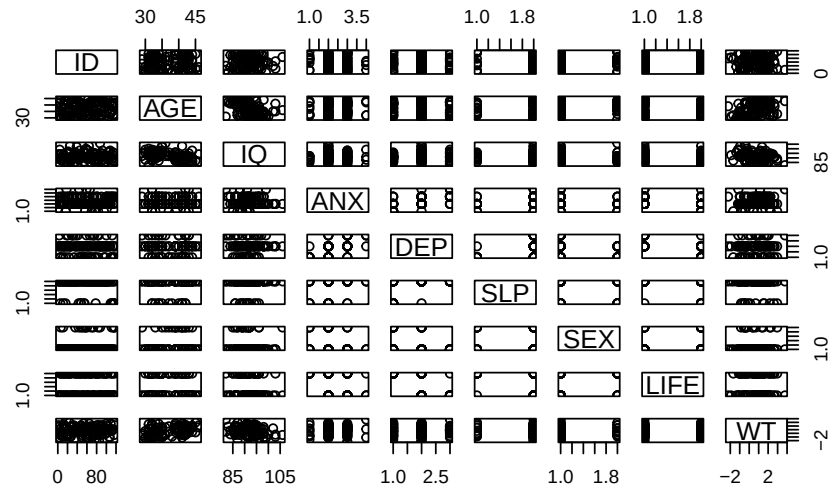
```
data:  IQ and WT
t = -3.0192, df = 98, p-value = 0.003231
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 -0.4616804 -0.1010899
sample estimates:
      cor
-0.2917158
```

With some functions you can pass an entire data.frame rather than a list of variables:

```
cor(fem, use = "pairwise.complete.obs")
pairs(fem)
```

	ID	AGE	IQ	ANX	DEP
ID	1.00000000	0.03069077	0.0370598672	-0.02941825	-0.0554147209
AGE	0.03069077	1.00000000	-0.4345435680	0.06734300	-0.0387049246
IQ	0.03705987	-0.43454357	1.0000000000	-0.02323787	-0.0001307404
ANX	-0.02941825	0.06734300	-0.0232378691	1.00000000	0.5437946347
DEP	-0.05541472	-0.03870492	-0.0001307404	0.54379463	1.0000000000
SLP	-0.07268743	0.02606547	0.0812993104	0.22317875	0.5248724551
SEX	0.08999634	0.10609216	-0.0536558660	-0.21062493	-0.3058422258
LIFE	-0.05604349	-0.10300193	-0.0915396469	-0.34211268	-0.6139017253
WT	0.02640131	0.41574411	-0.2917157832	0.11817532	0.0233742465
	SLP	SEX	LIFE	WT	
ID	-0.072687434	0.08999634	-0.05604349	0.026401310	
AGE	0.026065468	0.10609216	-0.10300193	0.415744109	
IQ	0.081299310	-0.05365587	-0.09153965	-0.291715783	
ANX	0.223178752	-0.21062493	-0.34211268	0.118175321	
DEP	0.524872455	-0.30584223	-0.61390173	0.023374247	
SLP	1.000000000	-0.29053971	-0.35186578	-0.009259774	
SEX	-0.290539709	1.00000000	0.22316967	-0.027826514	
LIFE	-0.351865775	0.22316967	1.00000000	-0.058605326	
WT	-0.009259774	-0.02782651	-0.05860533	1.000000000	

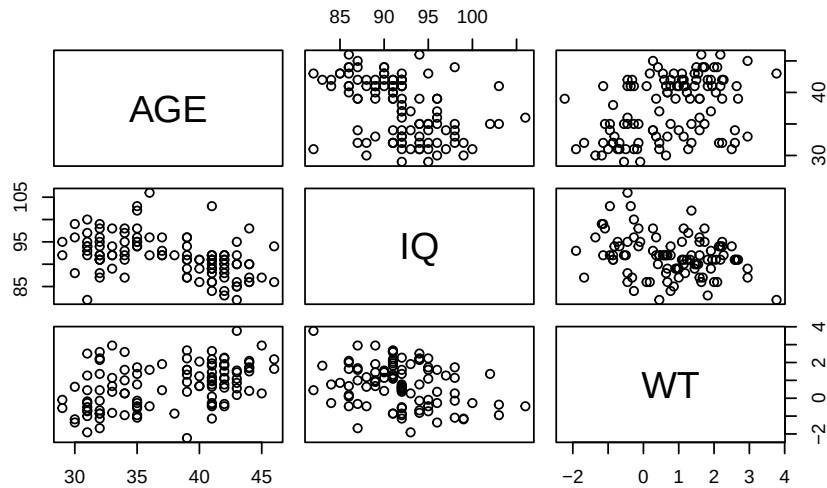
5. Getting acquainted with R



The output can be a little confusing particularly if it includes categorical or record identifying variables. To avoid this we can create a new object that contains only the columns we are interested in using the column binding `cbind()` function:

```
newfem <- cbind(AGE, IQ, WT)
cor(newfem, use = "pairwise.complete.obs")
pairs(newfem)
```

	AGE	IQ	WT
AGE	1.0000000	-0.4345436	0.4157441
IQ	-0.4345436	1.0000000	-0.2917158
WT	0.4157441	-0.2917158	1.0000000



When we have finished with the `newfem` object we can delete it:

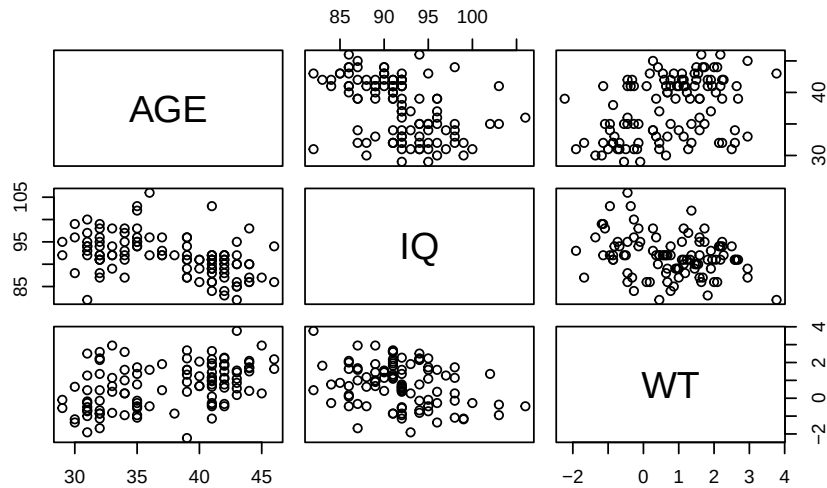
```
rm(newfem)
```

There was no real need to create the `newfem` object as we could have fed the output of the `cbind()` function directly to the `cor()` or `pairs()` function:

```
cor(cbind(AGE, IQ, WT), use = "pairwise.complete.obs")
pairs(cbind(AGE, IQ, WT))
```

5. *Getting acquainted with R*

	AGE	IQ	WT
AGE	1.0000000	-0.4345436	0.4157441
IQ	-0.4345436	1.0000000	-0.2917158
WT	0.4157441	-0.2917158	1.0000000

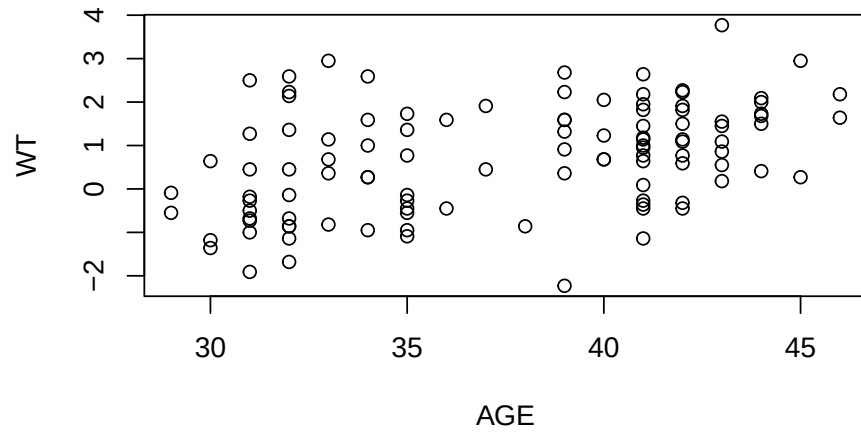


It is, however, easier to work with the `newfem` object rather than having to retype the `cbind()` function. This is particularly true if you wanted to continue with an analysis of just the three variables.

The relationship between `AGE` and `WT` can be plotted using the `plot()` function:

```
plot(AGE, WT)
```

5. Getting acquainted with R



And tested using the `cor()` and `cor.test()` functions:

```
cor(AGE, WT, use = "pairwise.complete.obs")
```

```
[1] 0.4157441
```

```
cor.test(AGE, WT)
```

Pearson's product-moment correlation

data: AGE and WT

t = 4.6841, df = 105, p-value = 8.457e-06

alternative hypothesis: true correlation is not equal to 0

95 percent confidence interval:

0.2452434 0.5612979

sample estimates:

cor

0.4157441

5. Getting acquainted with R

Or by using the linear modelling `lm()` function:

```
summary(lm(WT ~ AGE))
```

Call:

```
lm(formula = WT ~ AGE)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-3.10678	-0.85922	-0.05453	0.71434	2.70874

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-3.25405	0.85547	-3.804	0.00024 ***
AGE	0.10592	0.02261	4.684	8.46e-06 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.128 on 105 degrees of freedom
(11 observations deleted due to missingness)

Multiple R-squared: 0.1728, Adjusted R-squared: 0.165

F-statistic: 21.94 on 1 and 105 DF, p-value: 8.457e-06

We use the `summary()` function here to extract summary information from the output of the `lm()` function.

It is often more useful to use `lm()` to create an object:

```
fem.lm <- lm(WT ~ AGE)
```

And use the output in other functions:

```
summary(fem.lm)
```

Call:

```
lm(formula = WT ~ AGE)
```

Residuals:

Min	1Q	Median	3Q	Max
-3.10678	-0.85922	-0.05453	0.71434	2.70874

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-3.25405	0.85547	-3.804	0.00024 ***
AGE	0.10592	0.02261	4.684	8.46e-06 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.128 on 105 degrees of freedom

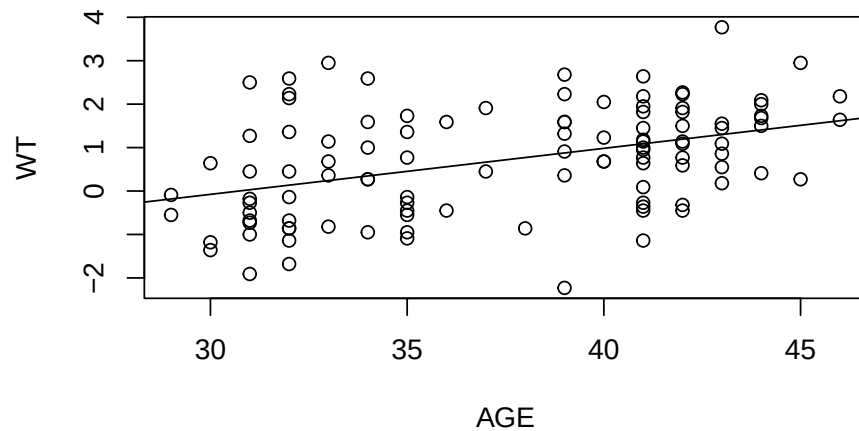
(11 observations deleted due to missingness)

Multiple R-squared: 0.1728, Adjusted R-squared: 0.165

F-statistic: 21.94 on 1 and 105 DF, p-value: 8.457e-06

5. Getting acquainted with R

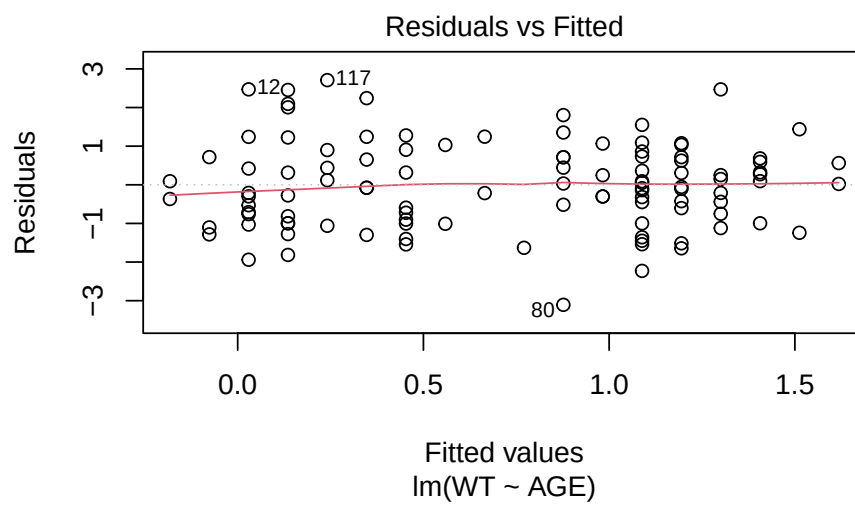
```
plot(AGE, WT)
abline(fem.lm)
```



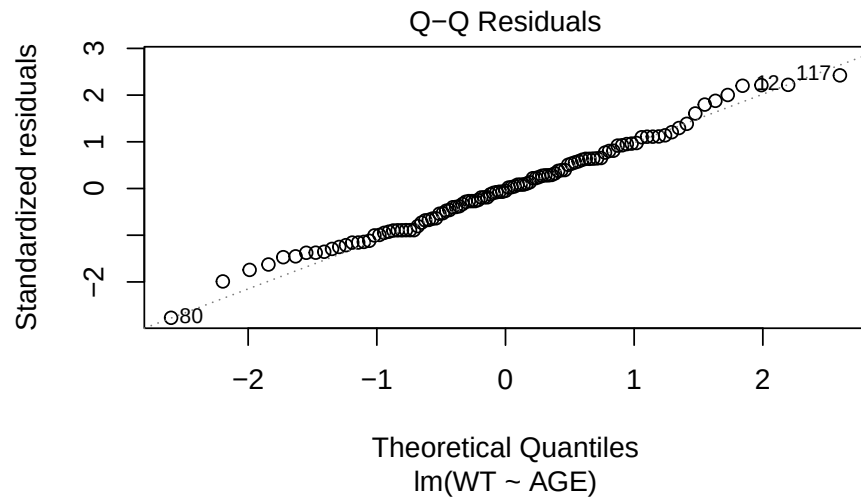
In this case we are passing the intercept and slope information held in the `fem.lm` object to the `abline()` function which draws a regression line. The `abline()` function adds to an existing plot. This means that you need to keep the scatter plot of AGE and WT open before issuing the `abline()` function call.

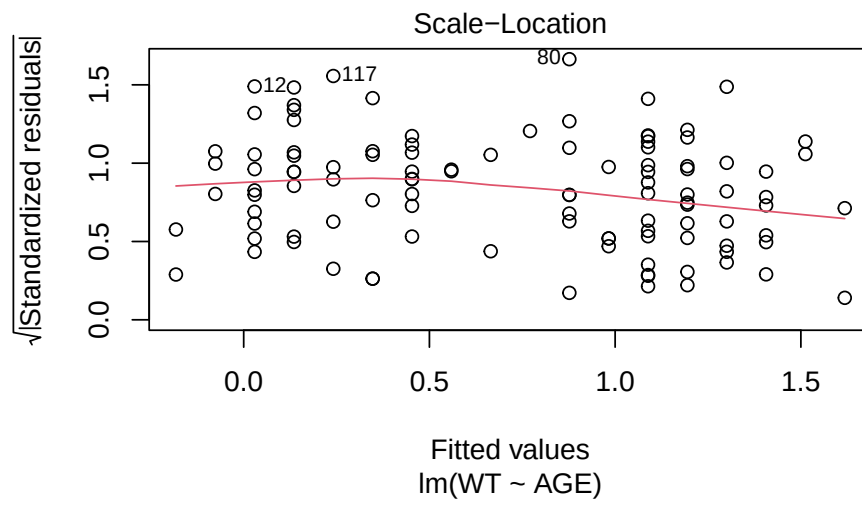
A useful function to apply to the `fem.lm` object is `plot()` which produces diagnostic plots of the linear model:

```
plot(fem.lm)
```

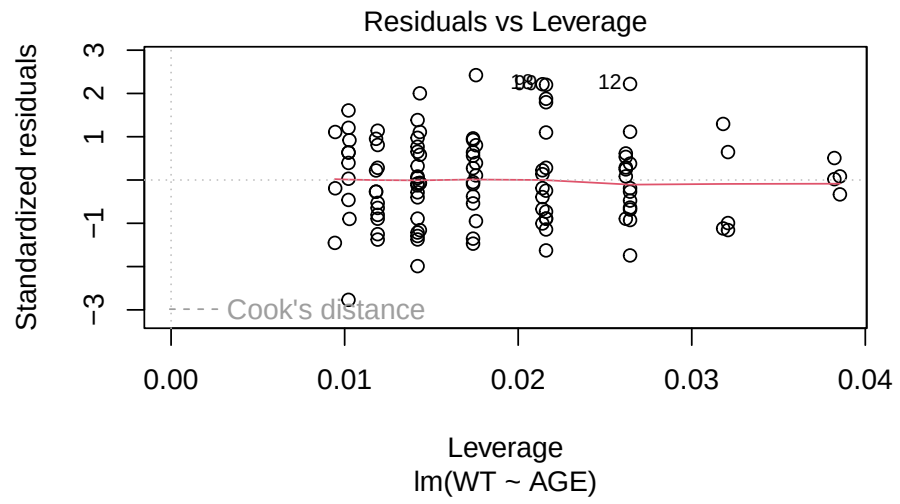


5. *Getting acquainted with R*





5. Getting acquainted with R

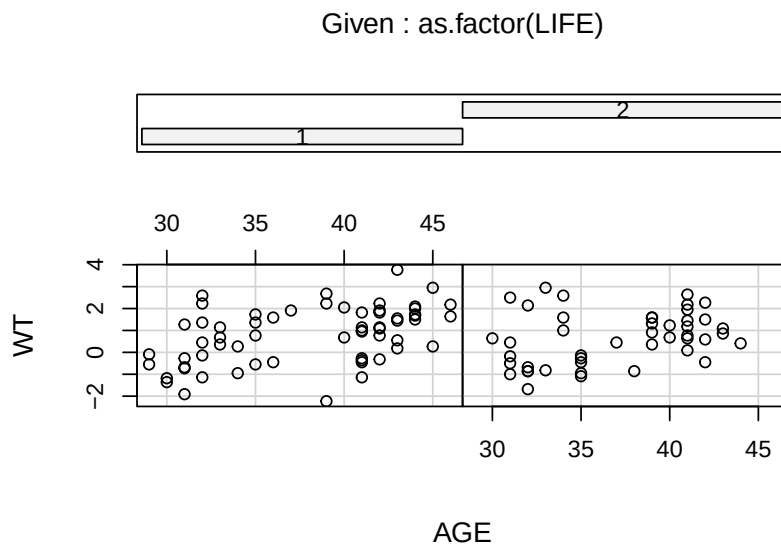


Objects created by the `lm()` function (or any of the modelling functions) can use up a lot of memory so we should remove them when we no longer need them:

```
rm(fem.lm)
```

It might be interesting to see whether a similar relationship exists between AGE and WT for those who have and have not considered suicide. This can be done using the `coplot()` function:

```
coplot(WT ~ AGE | as.factor(LIFE))
```

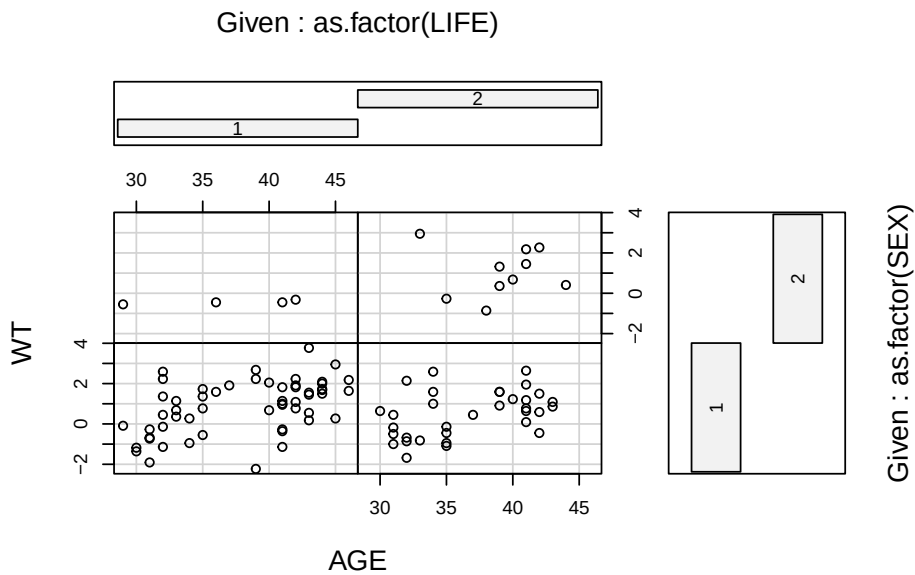


5. *Getting acquainted with R*

Missing rows: 21, 22, 31, 43, 44, 45, 69, 81, 101, 104, 114, 115

The two plots looks similar. We could also use `coplot()` to investigate the relationship between AGE and WT for categories of both LIFE and SEX:

```
coplot(WT ~ AGE | as.factor(LIFE) * as.factor(SEX))
```



Missing rows: 12, 17, 21, 22, 31, 43, 44, 45, 66, 69, 81, 101, 104, 105, 114, 115

although the numbers are too small for this to be useful here.

We used the `as.factor()` function with the `coplot()` function to ensure that R was aware that the LIFE and SEX columns hold categorical data.

5. *Getting acquainted with R*

We can check the way variables are stored using the `data.class()` function:

```
data.class(fem$SEX)
```

```
[1] "numeric"
```

5.1. Summary

The parameters of most R functions have default values. These are usually the most used and most useful parameter values for each function. The `cor.test()` function, for example, calculates *Pearson's product moment correlation coefficient* by default. This is an appropriate measure for data from a bivariate normal distribution. The DEP and ANX variables contain ordered data. An appropriate measure of correlation between DEP and ANX is *Kendall's tau*. This can be obtained using:

```
cor.test(DEP, ANX, method = "kendall")
```

```
Kendall's rank correlation tau
```

```
data: DEP and ANX
z = 5.5606, p-value = 2.689e-08
alternative hypothesis: true tau is not equal to 0
sample estimates:
      tau
0.4950723
```

5.1. Summary

- R is a functional system. Everything is done by calling functions.
- R provides a large set of functions for descriptive statistics, charting, and statistical inference.
- Functions can be chained together so that the output of one function is the input of another function.

5. *Getting acquainted with R*

- R is an object oriented system. We can use functions to create objects that can then be manipulated or passed to other functions for subsequent analysis.

6. Writing functions

For this topic, we will use data on weight and height to calculate body mass index. As a refresher, body mass index is calculated as follows:

$$\text{Body mass index} = \frac{\text{weight (kgs)}}{\text{height (m)}^2}$$

For this topic on writing functions in R, we will use BMI as an example to explore and demonstrate how we can create our own functions in R.

Let's say for example that you have been doing a research on children aged 11 years and older in 3 schools and you have collected the following data:

School 1516

school1516

	school	sex	ageMonths	weight	height
427	1516	1	138	24.5	126.0
428	1516	1	150	28.3	136.3
429	1516	1	162	32.2	143.5
430	1516	1	162	32.7	143.5
431	1516	1	150	28.6	137.0
432	1516	2	138	26.5	134.0
433	1516	1	150	29.9	139.2
434	1516	1	150	30.0	139.5
435	1516	1	162	34.0	148.0
436	1516	1	138	25.4	135.7

6. Writing functions

437	1516	1	150	32.3	143.0
438	1516	2	174	38.3	153.5
439	1516	2	162	41.6	151.0
440	1516	1	150	30.7	145.0
441	1516	2	186	46.8	155.2
442	1516	1	186	46.6	163.4
443	1516	1	150	33.5	145.5
444	1516	1	186	47.0	164.0
445	1516	1	174	41.1	159.5
446	1516	2	162	39.1	152.2
447	1516	2	174	40.9	155.5
448	1516	2	162	39.7	153.0
449	1516	2	162	40.9	153.2
450	1516	1	150	34.2	147.5
451	1516	2	150	41.8	149.4
452	1516	1	138	28.0	141.5
453	1516	1	138	30.0	142.0
454	1516	1	138	33.1	142.0
455	1516	1	186	46.1	167.5
456	1516	1	150	36.2	149.0
457	1516	2	162	47.4	156.0
458	1516	1	150	30.3	150.2
459	1516	2	150	36.4	152.1
460	1516	2	150	36.4	155.0
461	1516	2	150	44.1	155.0
462	1516	2	162	42.3	160.1
463	1516	2	179	50.4	163.5
464	1516	1	150	37.6	155.0
465	1516	2	138	36.0	154.5
466	1516	2	138	46.1	156.0

School 1522

school1522

	school	sex	ageMonths	weight	height
646	1522	1	203	30.6	140.5
647	1522	1	174	30.8	140.0
648	1522	1	162	29.3	136.3
649	1522	1	150	24.0	132.0
650	1522	1	150	28.1	132.1
651	1522	2	150	27.2	134.9
652	1522	1	162	34.2	139.2
653	1522	1	150	25.5	134.2
654	1522	1	138	24.6	129.0
655	1522	1	174	36.4	147.5
656	1522	1	150	28.7	137.5
657	1522	1	186	45.8	155.6
658	1522	1	174	36.3	151.6
659	1522	1	150	31.0	139.5
660	1522	1	138	29.0	134.3
661	1522	1	179	38.3	155.5
662	1522	2	138	31.3	138.4
663	1522	1	162	36.5	148.8
664	1522	1	155	36.8	145.2
665	1522	1	138	28.3	136.8
666	1522	1	138	26.8	137.3
667	1522	2	138	32.6	141.4
668	1522	2	138	31.9	143.0
669	1522	1	174	42.6	160.7
670	1522	2	198	57.8	158.0
671	1522	2	162	43.9	153.5
672	1522	2	150	35.1	150.6
673	1522	2	186	52.6	159.6
674	1522	2	150	45.1	152.8
675	1522	2	138	34.6	147.2
676	1522	2	150	45.3	153.1
677	1522	1	186	51.8	170.2
678	1522	2	150	57.1	154.2

6. Writing functions

679	1522	2	138	33.5	149.2
680	1522	1	150	36.3	154.1
681	1522	1	174	44.0	169.1
682	1522	2	150	44.5	158.3
683	1522	2	150	51.5	159.1
684	1522	2	138	47.4	157.8
685	1522	2	138	36.8	158.5
686	1522	2	138	52.0	161.0

School 1525

school1525

	school	sex	ageMonths	weight	height
752	1525	1	186	26.2	137
753	1525	1	186	32.7	138
754	1525	1	150	25.9	130
755	1525	1	162	30.4	137
756	1525	2	138	24.4	129
757	1525	2	138	23.8	130
758	1525	1	150	26.1	133
759	1525	1	150	26.4	135
760	1525	1	174	35.1	148
761	1525	1	162	28.7	142
762	1525	1	150	28.0	136
763	1525	1	174	34.0	149
764	1525	1	186	40.6	155
765	1525	2	150	35.8	142
766	1525	1	150	35.4	140
767	1525	2	138	27.8	137
768	1525	2	138	28.2	137
769	1525	2	138	29.7	139
770	1525	2	138	30.9	139

771	1525	1	138	28.2	137
772	1525	2	138	26.2	140
773	1525	2	138	26.6	140
774	1525	1	138	27.2	138
775	1525	2	138	27.0	141
776	1525	1	150	31.3	145
777	1525	2	162	33.9	152
778	1525	2	162	42.0	153
779	1525	2	185	38.3	157
780	1525	2	138	31.0	145
781	1525	2	138	32.3	145
782	1525	1	139	35.1	144
783	1525	2	150	36.4	152
784	1525	2	138	32.7	147
785	1525	1	174	44.9	166
786	1525	2	138	32.2	148
787	1525	2	138	36.4	148
788	1525	1	138	31.4	146
789	1525	2	138	45.0	149
790	1525	2	162	49.4	160
791	1525	2	138	34.3	150
792	1525	1	138	30.0	148
793	1525	2	150	37.0	156
794	1525	2	162	52.2	165
795	1525	2	138	42.9	158

In this dataset, the units of the height measurement is in centimetres.

Using what we have learned earlier on calculating BMI using R, I can perform the following R commands to get the BMI for each child in each of the schools:

```
## Calculate BMI for children in school 1516
school1516$weight / (school1516$height / 100) ^ 2
```

6. Writing functions

```
## Calculate BMI for children in school 1516
school1522$weight / (school1516$height / 100) ^ 2

## Calculate BMI for children in school 1516
school1525$weight / (school1516$height / 100) ^ 2
```

Because the commands are repetitive, I can easily copy and paste my initial line of code to calculate BMI for children in school 1516 and then just change the object names accordingly to calculate the BMI for children in the two other schools.

When I run these lines of code, I get the following results:

```
[1] 15.43210 15.23333 15.63695 15.87976 15.23789 14.75830 15.43095 15.41604
[9] 15.52228 13.79349 15.79539 16.25481 18.24481 14.60166 19.42954 17.45347
[17] 15.82409 17.47472 16.15550 16.87903 16.91463 16.95929 17.42632 15.71962
[25] 18.72730 13.98444 14.87800 16.41539 16.43128 16.30557 19.47732 13.43083
[33] 15.73414 15.15088 18.35588 16.50280 18.85363 15.65036 15.08153 18.94313
```

```
Warning in school1522$weight/(school1516$height/100)^2: longer object length
not a multiple of shorter object length
```

```
[1] 19.27438 16.57903 14.22865 11.65487 14.97150 15.14814 17.65012 13.10363
[9] 11.23083 19.76704 14.03492 19.43787 15.92035 14.74435 12.03967 14.34481
[17] 14.78490 13.57079 14.46527 12.21679 11.08343 13.92627 13.59168 19.58058
[25] 25.89564 21.92561 17.40726 26.08609 16.07485 15.58488 18.61440 22.96095
[33] 24.68185 13.94381 15.10926 17.16604 16.64656 21.43600 19.85735 15.12163
[41] 32.75384
```

```
Warning in school1525$weight/(school1516$height/100)^2: longer object length
not a multiple of shorter object length
```

```
[1] 16.50290 17.60176 12.57755 14.76284 13.00016 13.25462 13.46983 13.56612
[9] 16.02447 15.58555 13.69260 14.42986 17.80624 17.02735 14.69670 10.41216
[17] 13.32058 11.04253 12.14611 12.17362 10.83529 11.36315 11.58914 12.41023
[25] 14.02307 16.93116 20.82920 18.99425 11.04923 14.54889 14.42308 16.13472
[33] 14.13479 18.68887 13.40271 14.20099 11.74611 18.73049 20.69522 14.09435
[41] 18.89645 19.91636 25.34934 20.83308
```

The calculation for the BMI of children in school 1516 seems to have completed without issues and a vector of BMI results have been produced. However, for school 1522 and school 1525, there is a warning saying:

```
## Warning in school1522$weight/(school1516$height)^2: longer object length is not a multiple
## of shorter object length
```

Although a result has been provided, the warning gives me an indication that something is not quite right with my calculation and when I inspect further, I notice that in my formula for school 1522 and for school 1525, my denominator is still using data for school 1516 and this is most likely what is causing the warning message.

So, to correct this I go back to my lines of code and edit the denominators for school 1522 and school 1525 as follows:

```
## Calculate BMI for children in school 1516
school1516$weight / (school1516$height / 100) ^ 2

## Calculate BMI for children in school 1516
school1522$weight / (school1522$height / 100) ^ 2

## Calculate BMI for children in school 1516
school1525$weight / (school1525$height / 100) ^ 2
```

which gives me:

6. Writing functions

```
[1] 15.43210 15.23333 15.63695 15.87976 15.23789 14.75830 15.43095 15.41604
[9] 15.52228 13.79349 15.79539 16.25481 18.24481 14.60166 19.42954 17.45347
[17] 15.82409 17.47472 16.15550 16.87903 16.91463 16.95929 17.42632 15.71962
[25] 18.72730 13.98444 14.87800 16.41539 16.43128 16.30557 19.47732 13.43083
[33] 15.73414 15.15088 18.35588 16.50280 18.85363 15.65036 15.08153 18.94313
```

```
[1] 15.50132 15.71429 15.77161 13.77410 16.10277 14.94669 17.65012 14.15908
[9] 14.78277 16.73082 15.18017 18.91674 15.79459 15.92991 16.07852 15.83937
[17] 16.34076 16.48493 17.45479 15.12217 14.21653 16.30492 15.59978 16.49597
[25] 23.15334 18.63150 15.47594 20.65000 19.31656 15.96837 19.32626 17.88178
[33] 24.01416 15.04898 15.28626 15.38741 17.75817 20.34543 19.03550 14.64837
[41] 20.06095
```

```
[1] 13.95919 17.17076 15.32544 16.19692 14.66258 14.08284 14.75493 14.48560
[9] 16.02447 14.23329 15.13841 15.31463 16.89906 17.75441 18.06122 14.81166
[17] 15.02477 15.37188 15.99296 15.02477 13.36735 13.57143 14.28271 13.58081
[25] 14.88704 14.67278 17.94182 15.53816 14.74435 15.36266 16.92708 15.75485
[33] 15.13258 16.29409 14.70051 16.61797 14.73072 20.26936 19.29687 15.24444
[41] 13.69613 15.20381 19.17355 17.18475
```

I now do not get the warning message and the expected length of BMI values for each school has now been produced.

From this short example above, we realise how tedious a task it is to type in the code above every time we need to calculate BMI. Also, it becomes even challenging to debug issues with the code because we have to review and edit (as needed) each iteration of the calculation to see where it may have gone wrong (especially when doing a cut and paste approach).

It would be better (and easier) to have a function that calculates and displays the BMI values automatically. Fortunately, R allows us to do just that.

The `function()` function allows us to create new functions in R with the following generic syntax:

```
function_name <- function(argument1, argument2, ...) {
  ## Your code here
}
```

Using this template/generic syntax, we apply it to create a function called `calculate_bmi` as follows:

```
calculate_bmi <- function(weight, height) {
  weight / height ^ 2
}
```

We now have a function for calculating and producing BMI values.

Let us now test it with our 3 sets of data:

School 1516

```
calculate_bmi(
  weight = school1516$weight,
  height = school1516$height / 100
)
```

```
[1] 15.43210 15.23333 15.63695 15.87976 15.23789 14.75830 15.43095 15.41604
[9] 15.52228 13.79349 15.79539 16.25481 18.24481 14.60166 19.42954 17.45347
[17] 15.82409 17.47472 16.15550 16.87903 16.91463 16.95929 17.42632 15.71962
[25] 18.72730 13.98444 14.87800 16.41539 16.43128 16.30557 19.47732 13.43083
[33] 15.73414 15.15088 18.35588 16.50280 18.85363 15.65036 15.08153 18.94313
```

School 1522

```
calculate_bmi(
  weight = school1522$weight,
  height = school1522$height / 100
)
```

6. Writing functions

```
[1] 15.50132 15.71429 15.77161 13.77410 16.10277 14.94669 17.65012 14.15908
[9] 14.78277 16.73082 15.18017 18.91674 15.79459 15.92991 16.07852 15.83937
[17] 16.34076 16.48493 17.45479 15.12217 14.21653 16.30492 15.59978 16.49597
[25] 23.15334 18.63150 15.47594 20.65000 19.31656 15.96837 19.32626 17.88178
[33] 24.01416 15.04898 15.28626 15.38741 17.75817 20.34543 19.03550 14.64837
[41] 20.06095
```

School 1525

```
calculate_bmi(
  weight = school1525$weight,
  height = school1525$height / 100
)
```

```
[1] 13.95919 17.17076 15.32544 16.19692 14.66258 14.08284 14.75493 14.48560
[9] 16.02447 14.23329 15.13841 15.31463 16.89906 17.75441 18.06122 14.81166
[17] 15.02477 15.37188 15.99296 15.02477 13.36735 13.57143 14.28271 13.58081
[25] 14.88704 14.67278 17.94182 15.53816 14.74435 15.36266 16.92708 15.75485
[33] 15.13258 16.29409 14.70051 16.61797 14.73072 20.26936 19.29687 15.24444
[41] 13.69613 15.20381 19.17355 17.18475
```

In our example here, the `calculate_bmi()` function helped a little bit in making the code to calculate BMI for each student in each school more efficient. But the efficiency that functions provide become more evident when you need to make more complex operations. For example, what if you need to get the mean BMI for students in each school? Without a function, we will have to do the following script for each school:

School 1516

```
## Calculate BMI for children in school 1516
bmi_school1516 <- school1516$weight / (school1516$height / 100) ^ 2

## Get the mean BMI for children in school 1516
```

```
mean_bmi_school1516 <- mean(bmi_school1516)
```

```
mean_bmi_school1516
```

```
[1] 16.28491
```

School 1522

```
## Calculate BMI for children in school 1522
```

```
bmi_school1522 <- school1522$weight / (school1522$height / 100) ^ 2
```

```
## Get the mean BMI for children in school 1522
```

```
mean_bmi_school1522 <- mean(bmi_school1522)
```

```
mean_bmi_school1522
```

```
[1] 16.89955
```

School 1525

```
## Calculate BMI for children in school 1525
```

```
bmi_school1525 <- school1525$weight / (school1525$height) ^ 2
```

```
## Get the mean BMI for children in school 1525
```

```
mean_bmi_school1525 <- mean(bmi_school1525)
```

```
mean_bmi_school1525
```

```
[1] 0.001564695
```

As the operations/calculations we want to perform become more complex, the copy and paste method becomes more and more tedious. With the function approach, we can use the following:

6. Writing functions

```
calculate_mean_bmi <- function(weight, height) {  
  bmi <- weight / height ^ 2  
  
  mean_bmi <- mean(bmi)  
  
  return(mean_bmi)  
}
```

Applying the function to the datasets, we get:

School 1516

```
calculate_mean_bmi(  
  weight = school1516$weight,  
  height = school1516$height / 100  
)
```

```
[1] 16.28491
```

School 1522

```
calculate_mean_bmi(  
  weight = school1522$weight,  
  height = school1522$height / 100  
)
```

```
[1] 16.89955
```

School 1525

```
calculate_mean_bmi(  
  weight = school1525$weight,  
  height = school1525$height / 100  
)
```

[1] 15.64695

Part III.

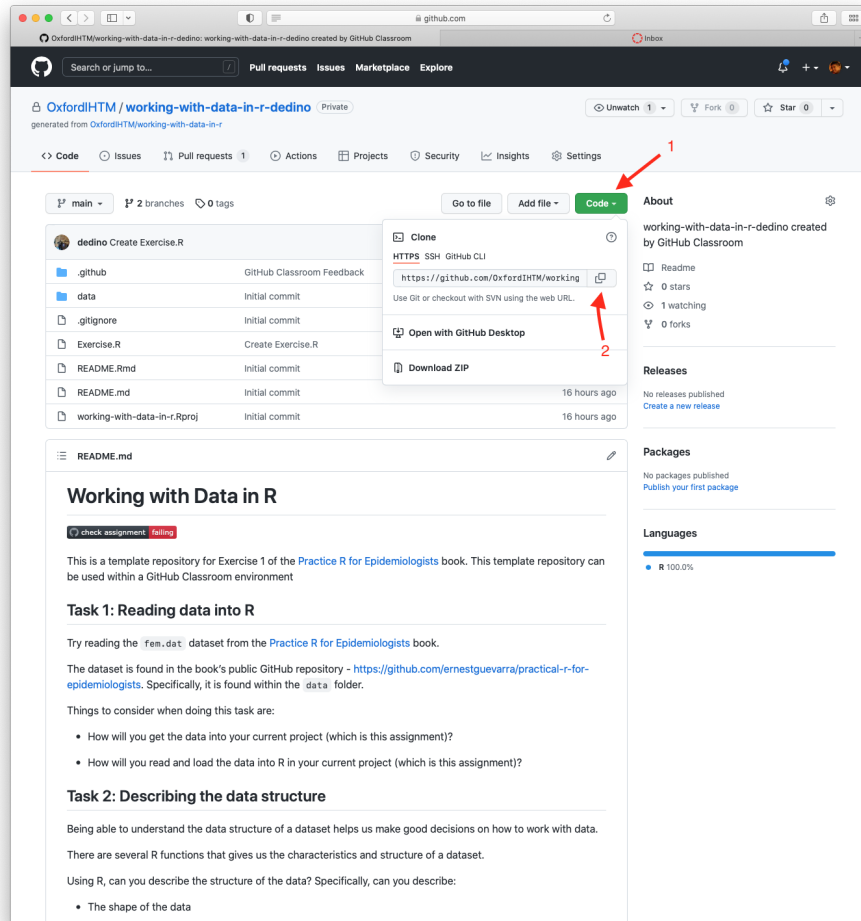
Processes

7. Cloning a GitHub repository into your local computer using RStudio

This tutorial is a summary of the the instructions described here.

7. Cloning a GitHub repository into your local computer using RStudio

7.1. Get the GitHub repository URL



7.1. Get the GitHub repository URL

1. Go to the repository's GitHub page

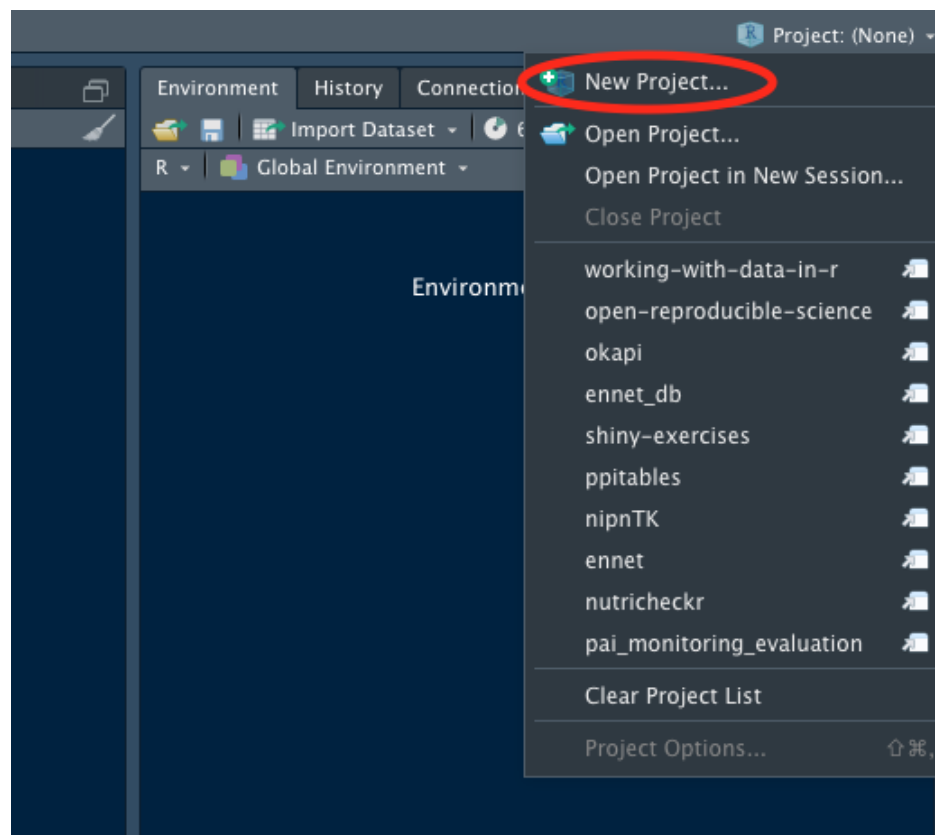
Click on the green button that is labeled code.

2. Copy the repository URL

Click on the copy to clipboard icon to copy the repository URL.

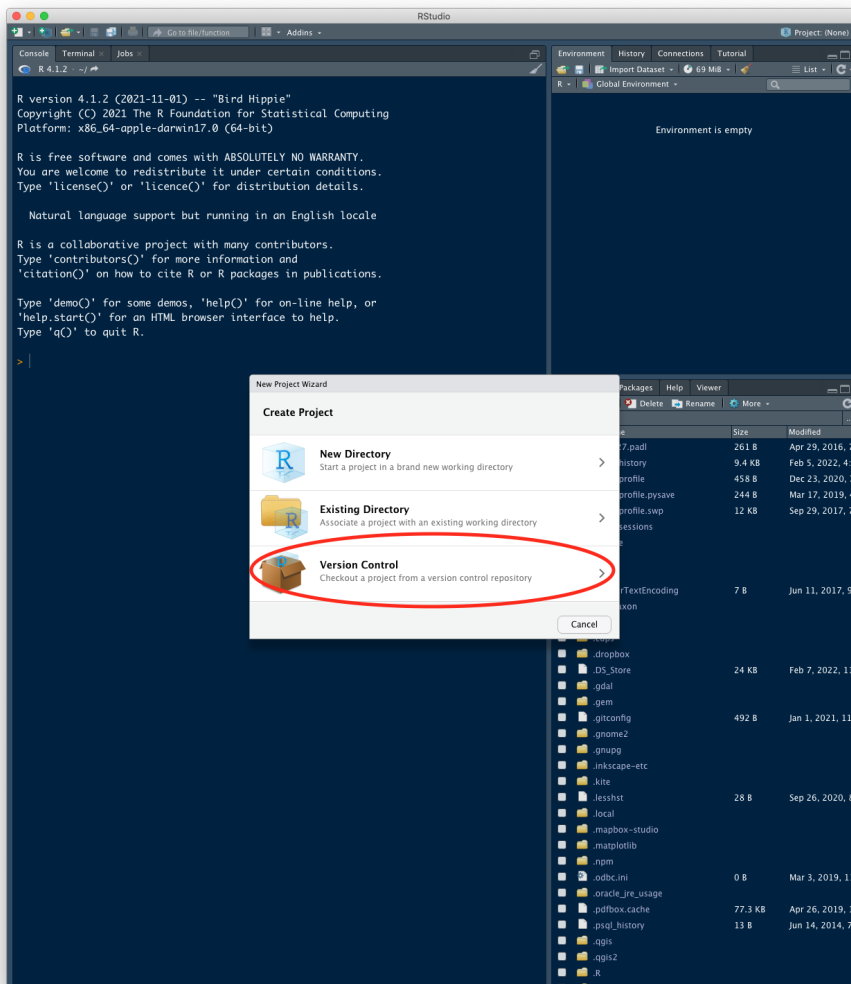
7. Cloning a GitHub repository into your local computer using RStudio

7.2. Go to RStudio and create new project



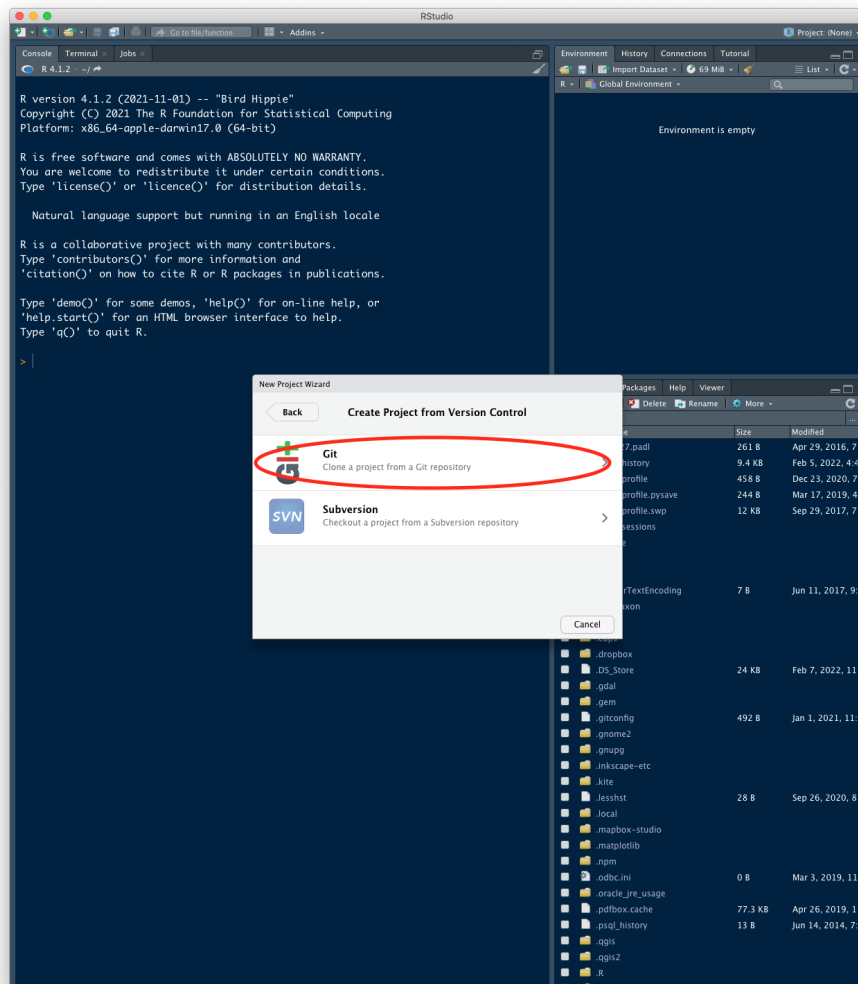
7.3. Choose Version Control

7.3. Choose Version Control



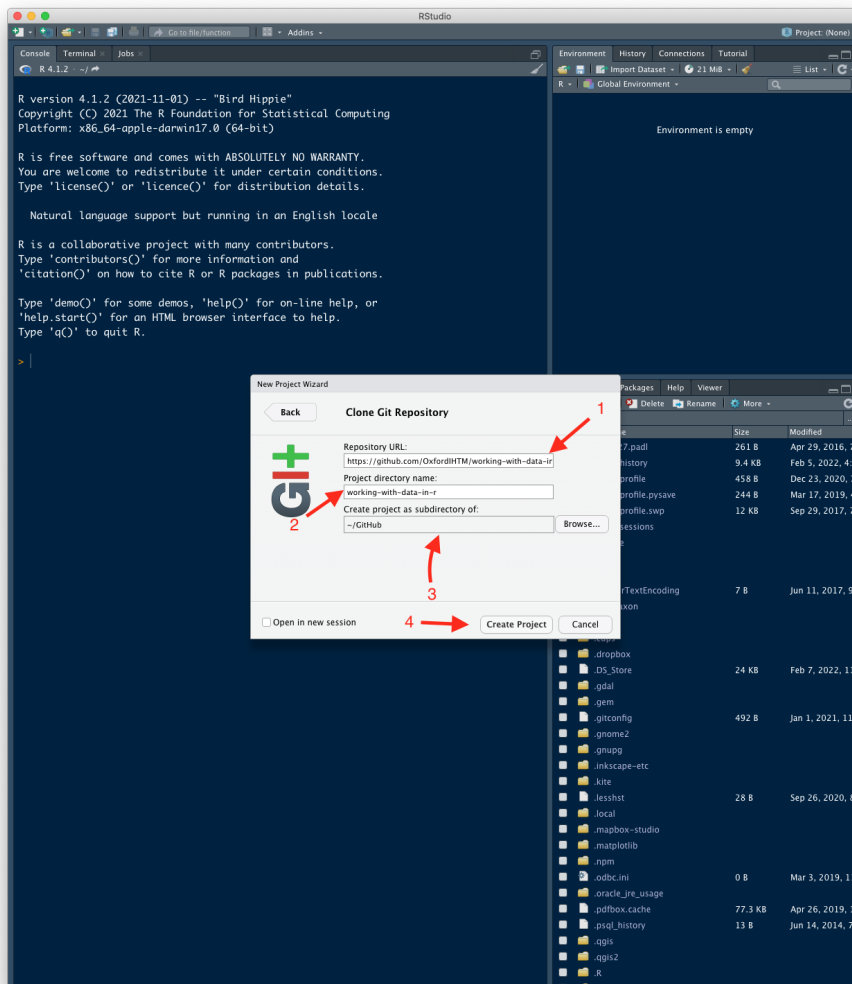
7. Cloning a GitHub repository into your local computer using RStudio

7.4. Select Git



7.5. Setup repository settings

7.5. Setup repository settings



7. Cloning a GitHub repository into your local computer using RStudio

1. Paste the repository URL you copied earlier

2. Set the project directory name

The project directory name should be specified already after you paste the repository URL. Use the suggested directory name.

3. Set local directory

Browse for the directory on your local computer where you want to save the files for the specified project.

4. Create project

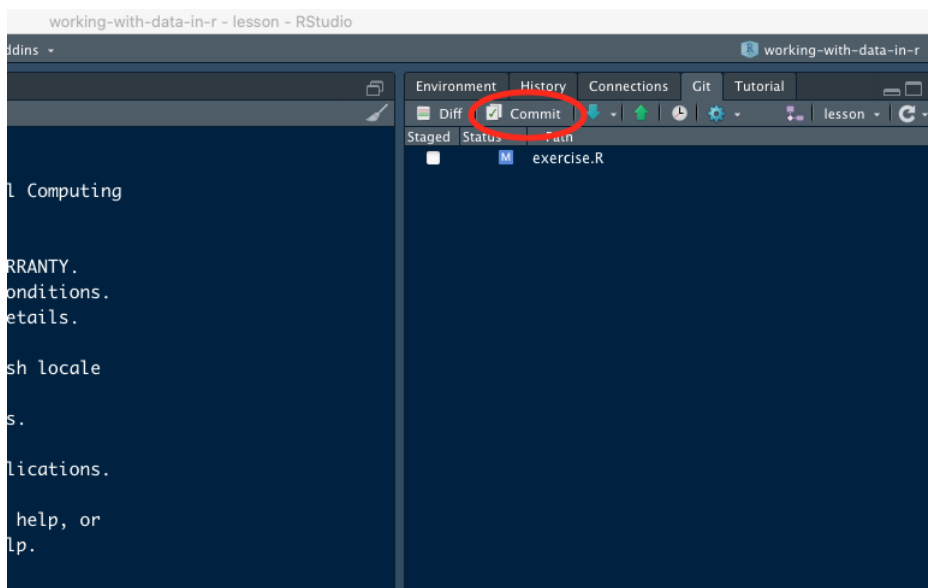
Click on the `Create Project` button/icon.

You will now have the GitHub repository in your local computer.

8. Committing your changes and pushing them to GitHub

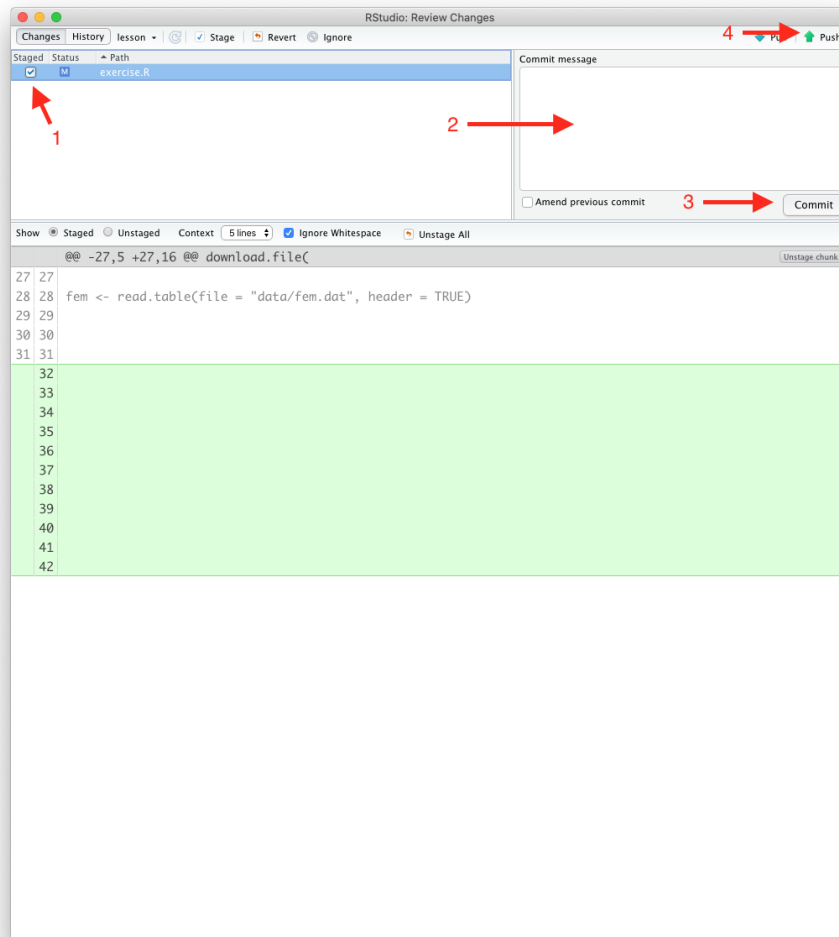
Following are the steps to take in committing your changes in RStudio and pushing them to GitHub.

8.1. Click on Commit in the Git tab on RStudio



8. Committing your changes and pushing them to GitHub

8.2. Getting changes saved and push to GitHub



8.2. *Getting changes saved and push to GitHub*

1. **Stage changes**

Tick the box beside the file that has changed to stage the changes.

2. **Add a commit message**

Every time you make a commit you must also write a short commit message.

Write a commit message in the **Commit message** dialog box. In the commit message, describe the changes that you made.

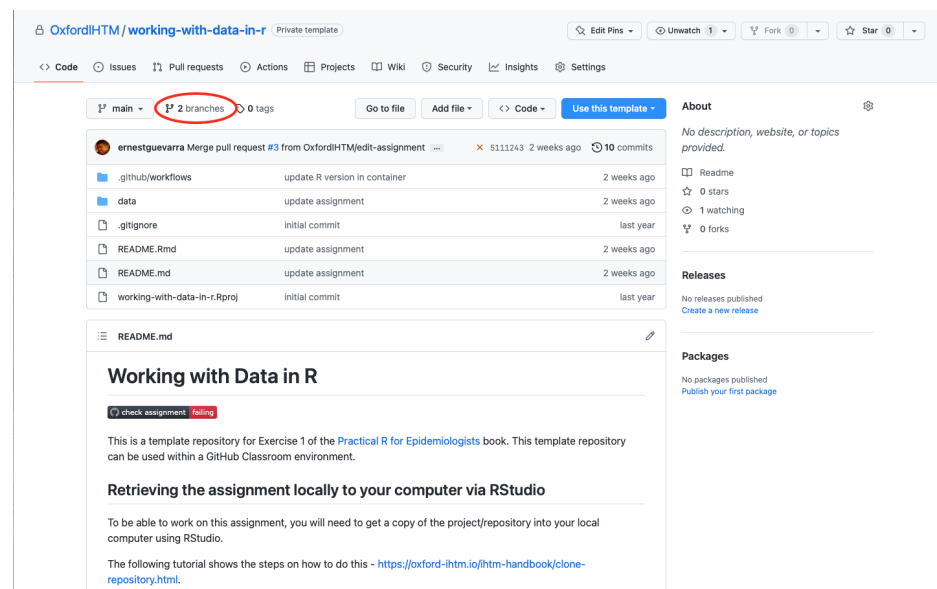
8. *Committing your changes and pushing them to GitHub*

3. Click on the **Commit** button

4. Click on the **Push** button

8.3. Initiate a pull request

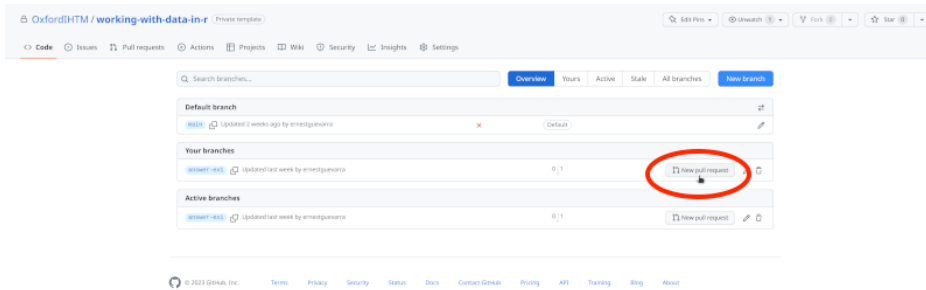
1. Click on the **branches** link from your repository



2. Make a pull request

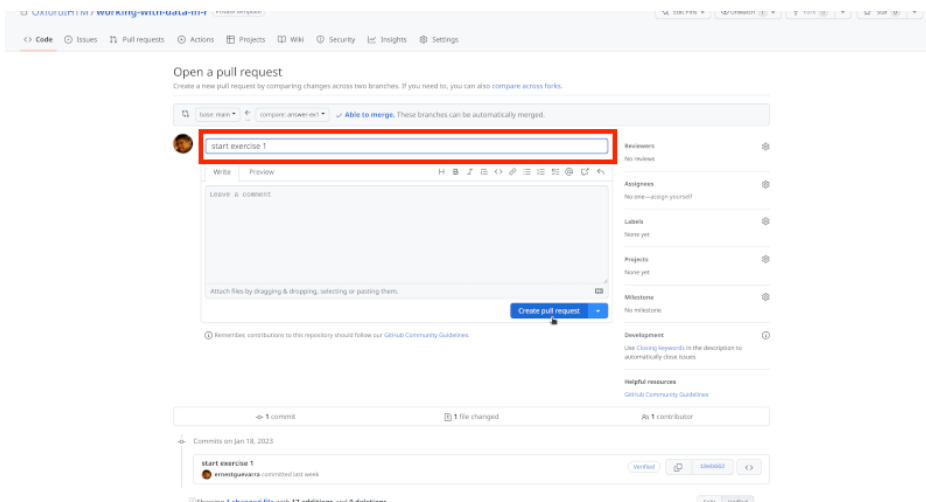
Click on the **Make pull request** link on the appropriate branch.

8.3. Initiate a pull request



<https://github.com/OxfordH1TM/working-with-data-in-r/compare/start-exercise-1>

3. Enter a title for your pull request

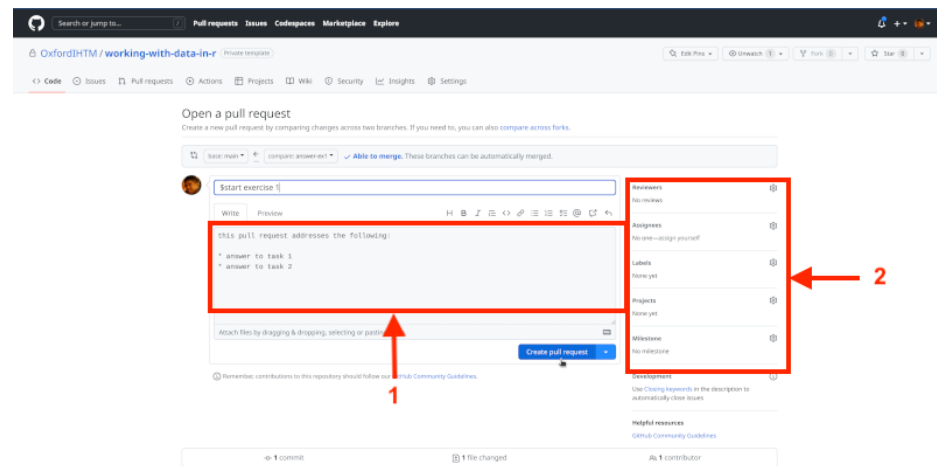


Make the title as short but as informative as possible.

8. Committing your changes and pushing them to GitHub

4. Create a pull request

Add further description about the pull request (optional) and then click on **Create pull request** button



i Note

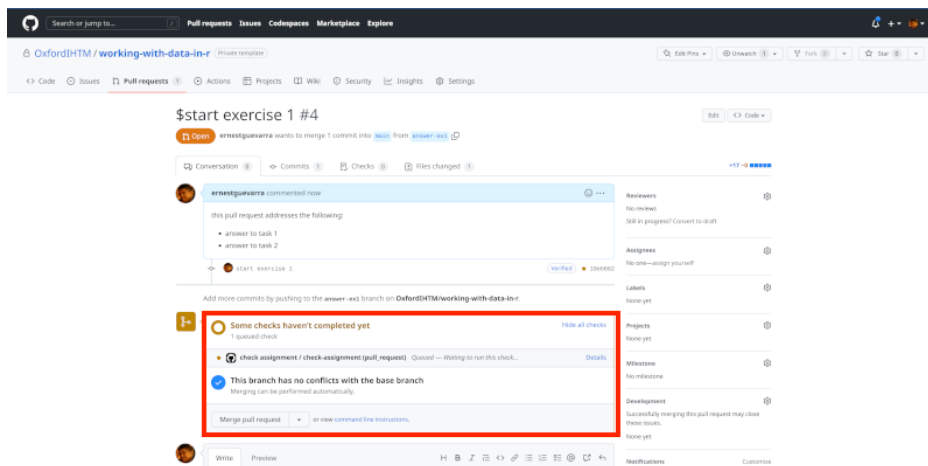
If you think more information will help the reviewer navigate through the changes you have made, use the comment box to add more details. This comments box can interpret Markdown syntax so you can format your text accordingly.

On the right hand side of the pull request page, you can set a specific reviewer for your pull request (recommended). Also, given that you are making this pull request, assign this pull request to you so you are notified of the progress of this pull request.

8.3. Initiate a pull request

5. Wait for review

If the project has automated checks included, you will see that these checks will get initiated.



If there are no issues with the code, the automated checks should show that all checks have passed.

8. Committing your changes and pushing them to GitHub

The screenshot shows a GitHub pull request titled "start exercise 1 #4". At the top, it indicates that user "ernestguevarra" wants to merge 1 commit into the `main` branch from the `answer-ex1` branch. Below this, there are tabs for Conversation (0), Commits (1), Checks (2), and Files changed (1). The main content area shows a commit history with three entries: "start exercise 1" (verified, 19eb662), "ernestguevarra closed this 31 minutes ago", and "ernestguevarra reopened this 9 minutes ago". A red box highlights a section at the bottom of the pull request, which contains two status checks: "All checks have passed" (1 successful check) and "This branch has no conflicts with the base branch" (Merging can be performed automatically). Below these checks is a "Merge pull request" button. To the right of the pull request, there are sections for Reviews (No reviews), Assignees (No one—assign yourself), Labels (None yet), Projects (None yet), Milestone (No milestone), Development (Successfully merging this pull request may close these issues), and Notifications (Unsubscribe). At the bottom, there is a "Write" button and a "Preview" button, along with a rich text editor toolbar.

i Note

Wait for reviewer's feedback/comments. If reviewer request's changes, make changes to your code and then commit and push again (as above). If your project has automated checks, this will get triggered again within the same pull request. Your reviewer will be notified of the changes you have made and should review your work again. Once reviewer approves changes, you can then merge your work to the main branch.

9. Participating in an existing R/RStudio project

Following are the general steps to take when participating in an existing R/RStudio project that has been initiated and led by someone else.

The following diagram illustrates the steps in this process:

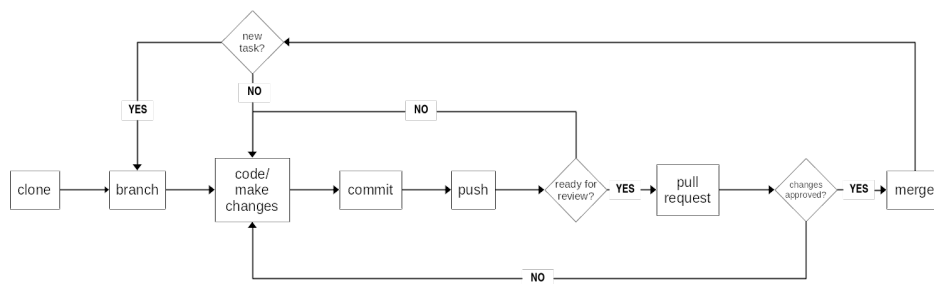


Figure 9.1.: A flowchart of a git-based development workflow

9.1. Clone the project to your local machine

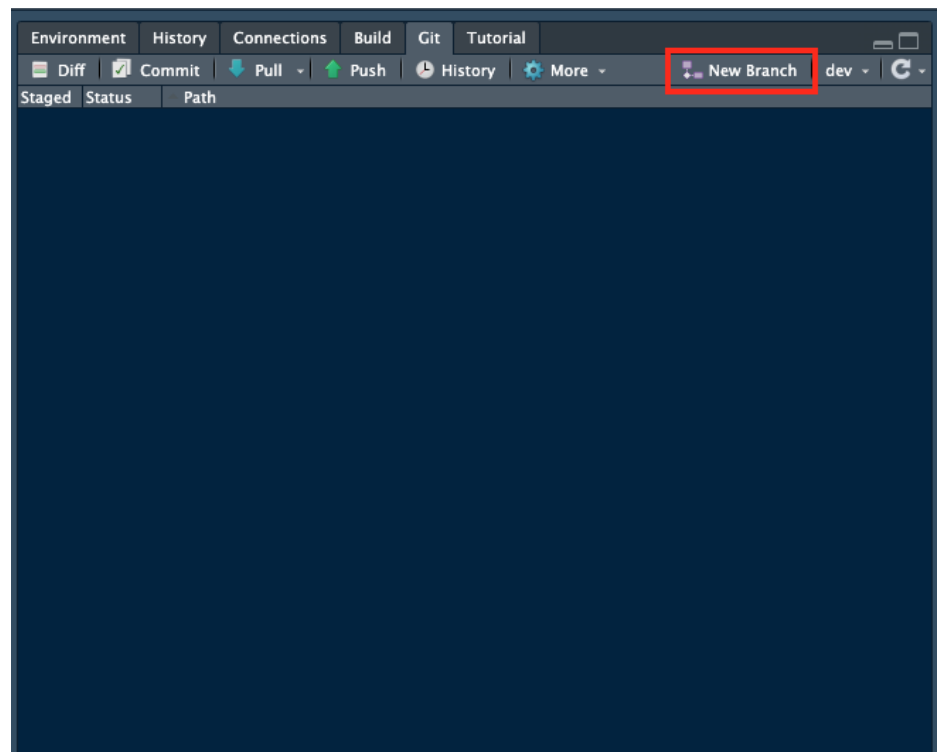
Steps in cloning a project to your local machine is described in Chapter 7.

9.2. Create a new branch from the main branch

Before making any changes to the project, create a new branch as follows:

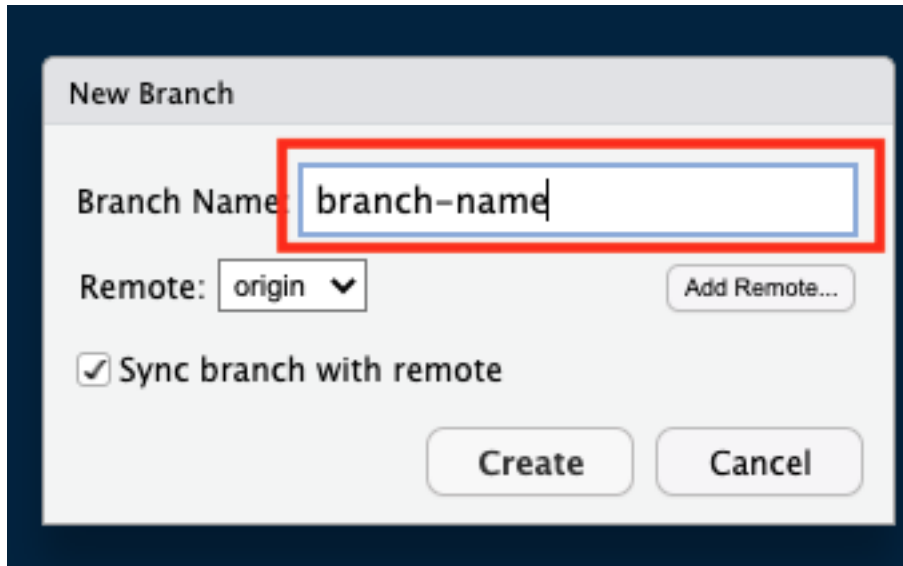
9. Participating in an existing R/RStudio project

9.2.1. Click on New Branch



9.2. Create a new branch from the main branch

9.2.2. Name the new branch

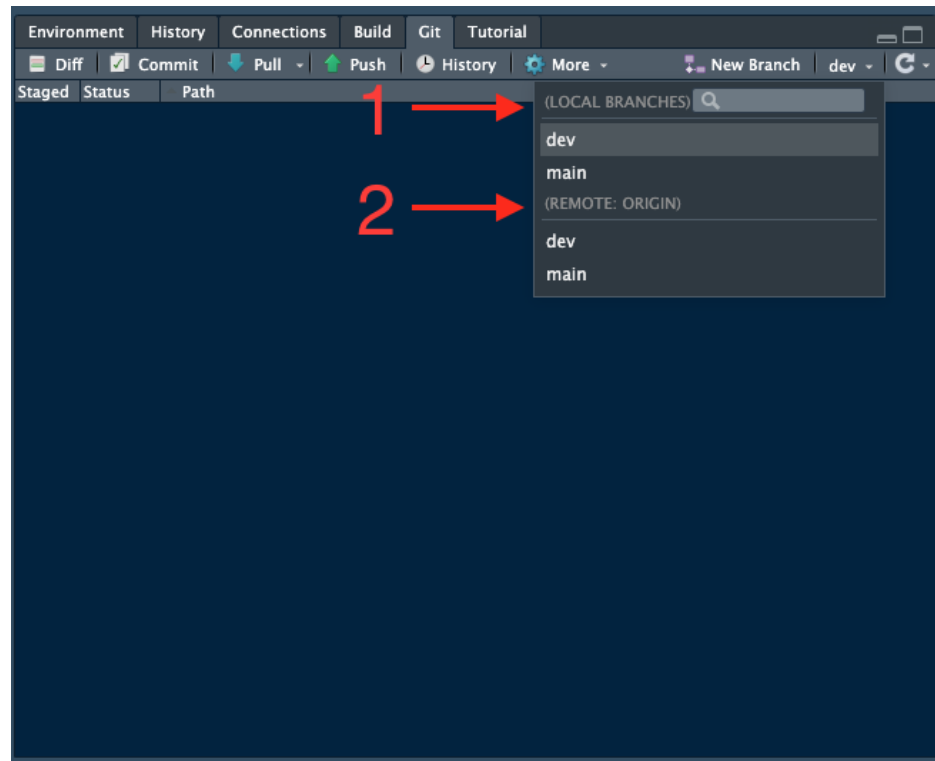


Name the branch uniquely. The best way to name a branch will be based on how the team/person you are working with prefers to name branches. Some would like the branch name to succinctly describe the type of change that is being made. Some may ask you to name your branch with your username. Some may ask you to name your branch using coded values.

Once named, click on **Create**

You will now see the new branch in the list of branches

9. Participating in an existing R/RStudio project



9.3. Code and make changes to your branch

Start coding and implement the changes you want to make or the changes that your collaborator/s asked you to make.

9.4. Commit and push your changes and initiate a pull request

After making changes, you should **commit and push** your changes. This process is described in Chapter 8. Your code and your changes do not have to be complete already for you to commit and push changes. It is good practice to commit and push frequently (at least once a day usually at the end of your coding session). See this as similar to saving your work at multiple stages.

Once your code and the changes you want to make are complete (and ideally that they are working correctly on your local machine), and that you are ready to have your work reviewed, you can now make a **pull request**. This process is also described in Chapter 8.

9.5. Merge pull request

Once your chosen reviewer has seen your work, they may ask you to make changes based on what they see with your code. If so, then start coding again on the same branch and address the reviewers comments, commit those changes and push the changes to your remote repository. Your changes will push into the same existing open pull request that is waiting approval. The reviewer can then view your changes and make the necessary feedback.

Once reviewer approves your changes, they may either merge your pull request themselves or they may let you know in their feedback that they are happy with your changes and that you can now merge your pull request. If so, then click on the **Merge pull request** button.

Your changes have now been integrated into the main branch of the project.

10. Initiating an R/RStudio project

Following is a diagram of the steps in initiating your own R/RStudio project.

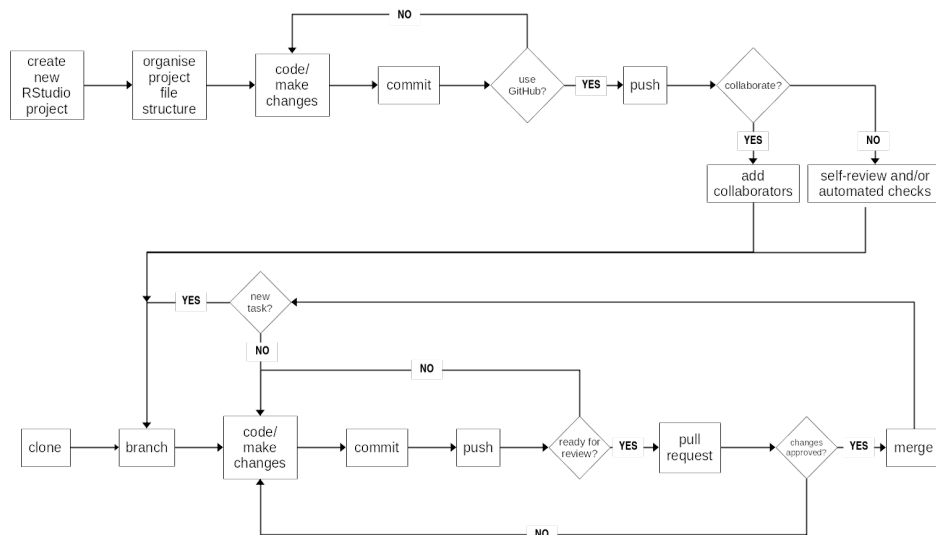
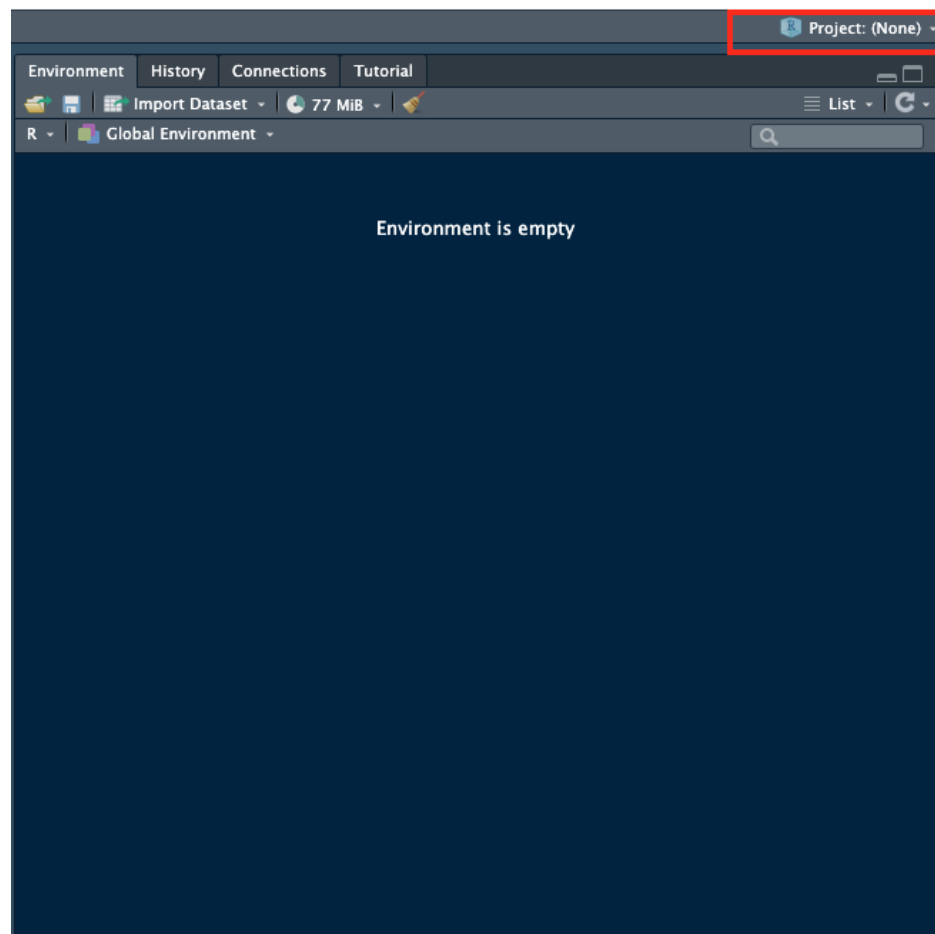


Figure 10.1.: A flowchart for initiating an R project

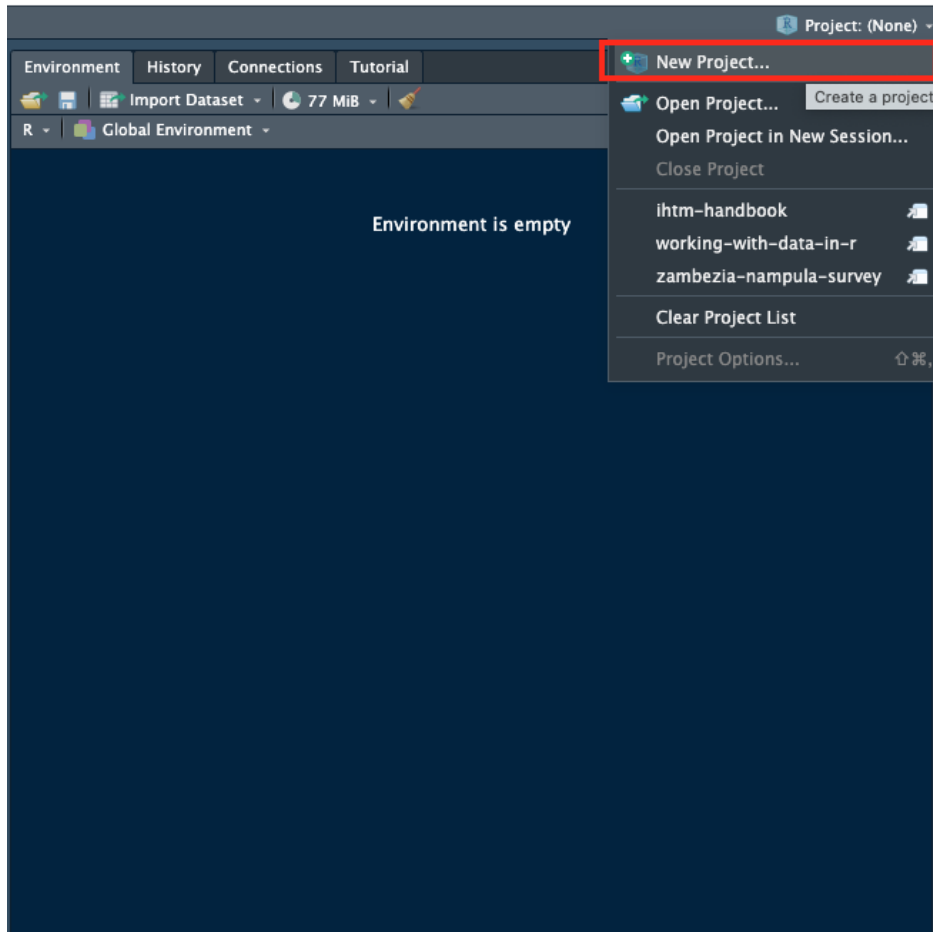
10. Initiating an R/RStudio project

10.1. Create a new project in RStudio

10.1.1. Click on New Project button on RStudio

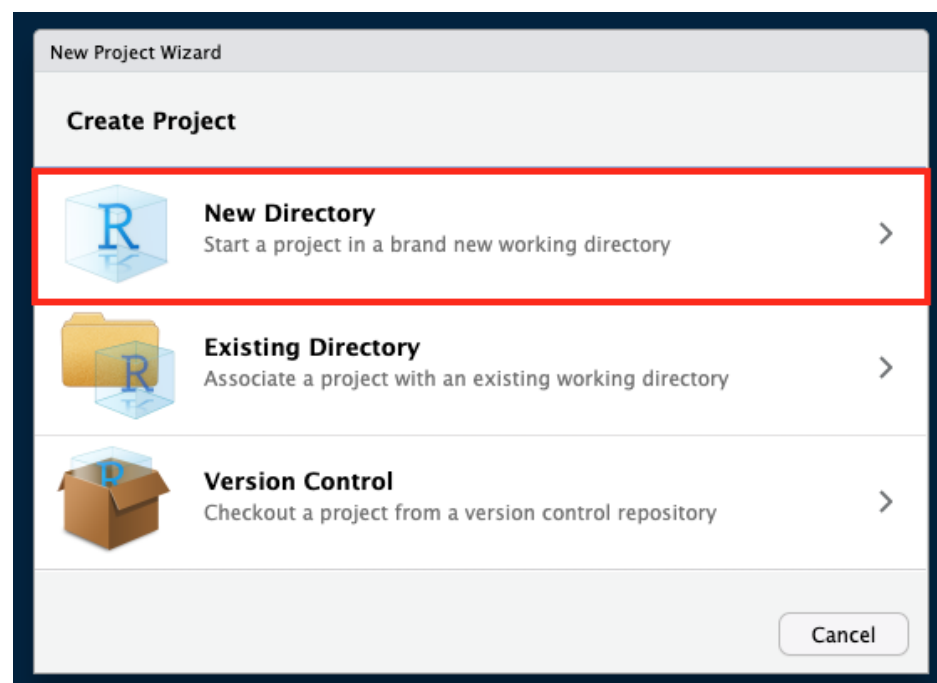


10.1. Create a new project in RStudio



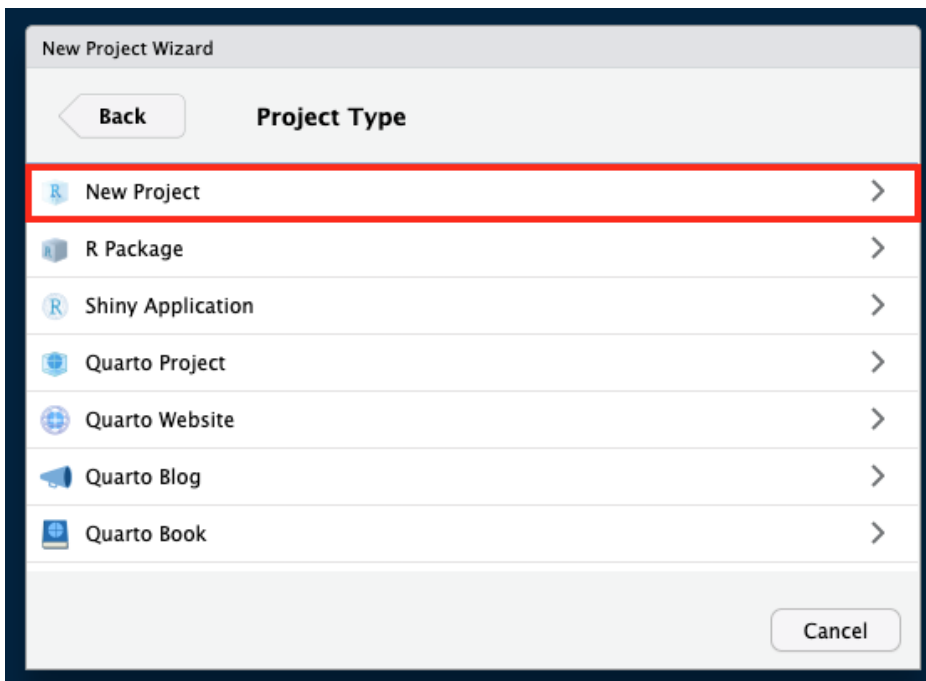
10. Initiating an R/RStudio project

10.1.2. Create a New Directory



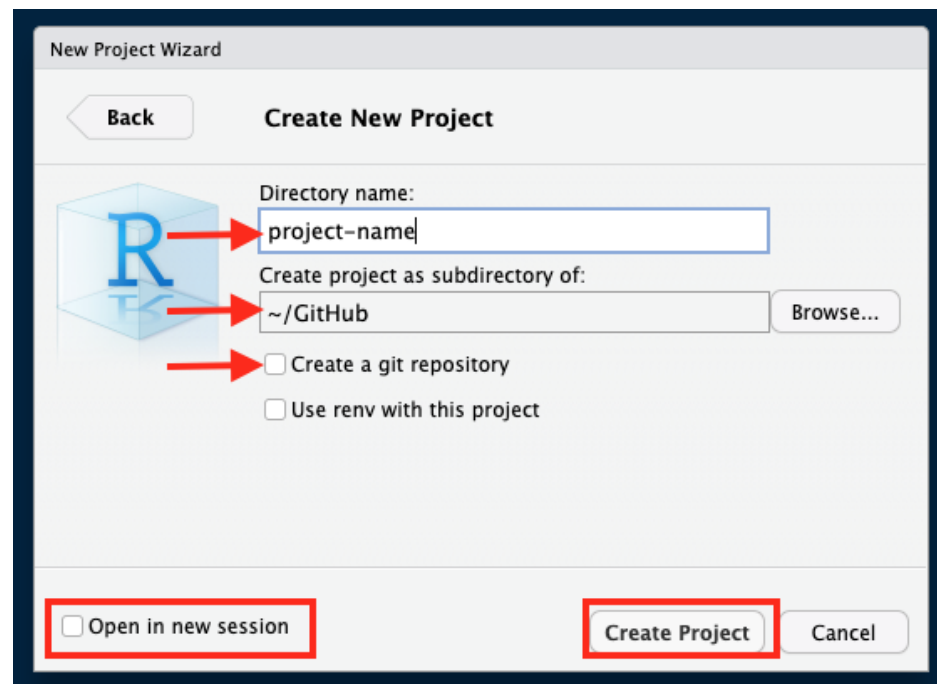
10.1. Create a new project in RStudio

10.1.3. Select New Project as project type



10. Initiating an R/RStudio project

10.1.4. Specify details for new project



Specify a project name

10.1. Create a new project in RStudio

i Note

Best practices for naming a project are:

- Make sure that name is succinct (as short as possible while at the same time descriptive of the project);
- Don't use spaces for your project name. If you need to separate words, use a *hyphen* or an *underscore*;
- Avoid using capital letters.

Specify a directory/location

Select a directory in your local machine where to place the directory of your new project

Decide whether to use git to version this project

Here you can decide whether you want to use git to version your project. Remember that using git doesn't mean you have to use GitHub. git is software installed in your local machine and it versions what you have on your local machine. You don't need GitHub or any other similar service to version your code with git in your local machine.

I would recommend that you tick this option for any new project you create so that you can version your work in your local machine even if you don't want or decide not to use GitHub or any other remote git service.

Do you want to open a new session

This is by default unticked and will open the new project within the existing RStudio session (if any). This means that if you have an existing RStudio session with another project that you are working on, that project will

10. Initiating an R/RStudio project

be closed and the new project you are creating will open in the existing RStudio session.

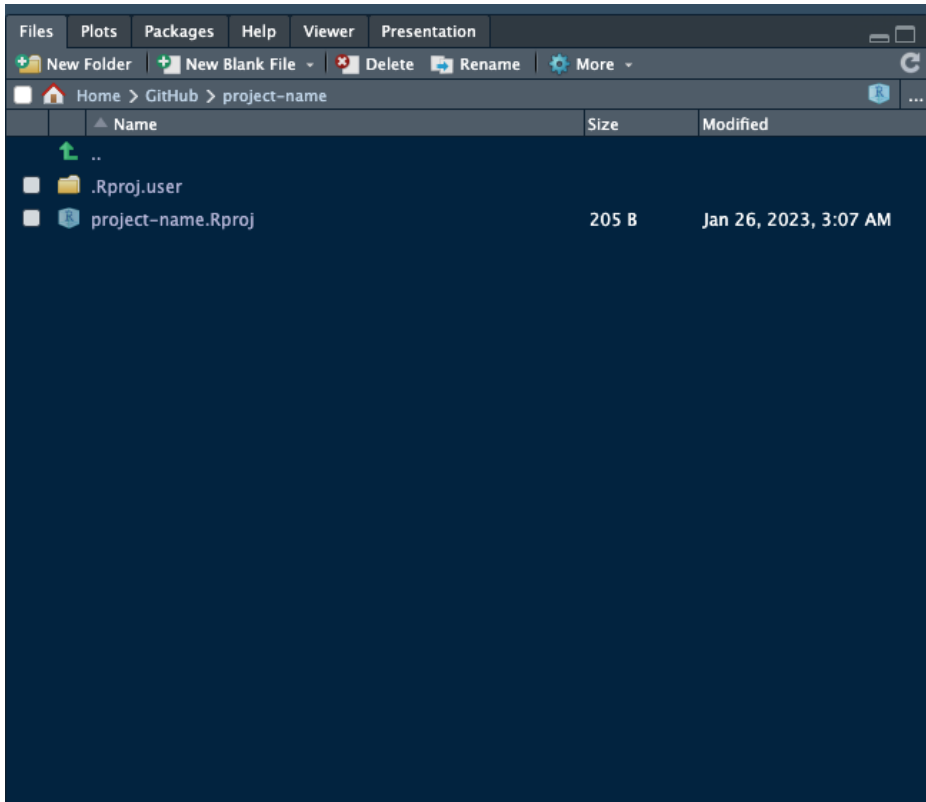
If you need your existing RStudio session and the project within it to remain open alongside the new project you are creating, tick this box/option.

Click on Create New Project

Once you click on *Create New Project*, you will now see the new project open in RStudio.

You will also see something like below within the file explorer pane of RStudio.

10.1. Create a new project in RStudio



10.2. 2. Structure/organise your new project appropriately

Note

Project organisation is vital because:

- supports productivity because the different components of the project are placed in directories where they should be;
- enables clarity in communicating project structure;
- facilitates collaboration.

Organising an R project can be user- and project-dependent but there are generally accepted project organising structure that is common to most well-organised projects. Below is an example:

```
|-- my-project
    |-- data
    |-- output
        |-- figures
    |-- R
    |-- my-project.Rproj
    |-- analysis_workflow.R
    |-- README.md
```

10.3. 3. Start coding

This will include creating bespoke R functions (as required) and creating an Rscript for the step-by-step processes in your scientific workflow.

10.4. Next steps

The next steps will depend on whether you will use git and GitHub for versioning your project and whether or not you will work on your project as a solo scientist or work and collaborate with other scientists.

11. Creating portable and reproducible scientific workflows

At this point, you would have written your own R code and R scripts and saved these within an R file (.R file extension).

By now, you would have also appreciated how extensible R is through built-in packages and/or through functions that you have created yourself.

So far, in the examples that we have worked on, the operations and the problems have been quite straightforward. But from your own experience dealing with your own data, real world data is far from straightforward and far from simple. Complexity is almost always a given.

R's scripting capability and R's extensibility are its main characteristics that make R a good tool for creating robust scientific workflows particularly for complex data and research projects.

A typical scientific workflow would have the following steps:

11. Creating portable and reproducible scientific workflows

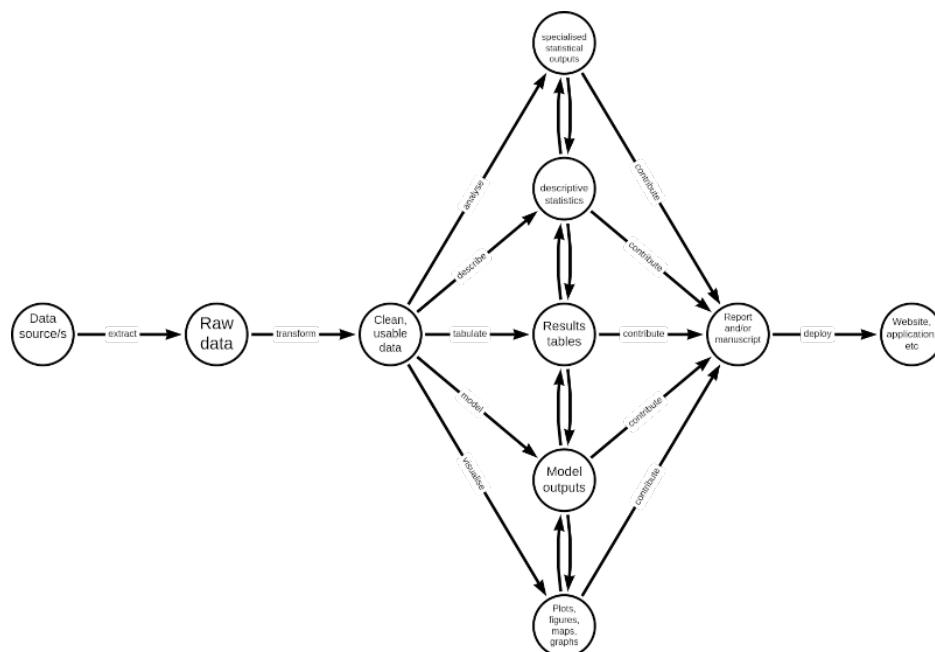


Figure 11.1.: A flowchart for an example scientific workflow

In general, an R script should reflect the different steps outlined above. Hence, an R script of a scientific workflow would tend to look like this:

```
## Load libraries
```

```
## Retrieve and read data
```

```
## Process data
```

```
## Analyse data
```

11.1. Create a new RStudio project

```
### Descriptive analysis

### Statistical tests

### Model specifications


## Outputs

### Tabulation of results

### Model outputs

### Plots


## Report
```

In this chapter, we will go through a step-by-step walkthrough of how to build a robust scientific workflow in R. A robust workflow is one that is ***portable*** i.e., not dependent on hardware and software and instead can be run on almost any machine with very minimal, if any, additional setup or configuration required, and one that is ***reproducible*** i.e., can be run over and over again without issues, providing the expected results with the same data or providing updated results with new and/or updated data.

11.1. Create a new RStudio project

The steps here are a summary of what is found in Section 10.1.

11. Creating portable and reproducible scientific workflows

Tip

- Open RStudio
- Click on the **File** option in the RStudio menu. In the dropdown menu, select **New Project**
- In the menu window, select **New directory** option.
- In the next menu window, select **New project** option.
- In the next menu window, enter the following details:
 - Name of the project - important to make the project name as short as possible but descriptive of the project you are creating; don't use spaces, instead use dash (or underscore) and avoid using capital letters;
 - Select the directory in your computer in which you want to save the project in. Click on **Browse** to open your computers file manager and navigate to the directory you want to save your project in;
 - Tick the selection box to make this project a git repository (whilst this is not necessary, this is highly recommended especially if you are collaborating with others);
 - Tick the selection box to enable **renv** in this project (this is what mainly contribute to the portability of your project); and,
 - Click on **Create project**

11.2. Create an R file for package dependencies

It is best practice to create a standalone R file specific for invoking/calling on R package dependencies. I recommend calling this file `packages.R` and this file should be saved in the root directory of the project you just created.

These are steps on how you can create this file.

Tip

Steps for creating an R package dependencies file:

- Click on the **File** option in the RStudio menu. In the dropdown menu, select **New File** and then in the next dropdown menu, select **R script**.
- A new tab will open in your text editor pane of RStudio (upper left pane) with the name *Untitled1*. Save this file by clicking on the disk icon on the text editor menu or do a keyboard shortcut with **CTRL + s**. Give this empty R script the filename `packages.R`.
- You should now see a file in the main directory/root directory of your project named `packages.R`
- Add code in the `packages.R` file specifying the packages you will be using in this project. There will be standard packages that we will always use with this type of workflow. So a template/generic `packages.R` file will contain the following:

```
#####  
#  
#'  
#' General packages needed for a targets workflow
```

11. Creating portable and reproducible scientific workflows

```
# '  
#  
#####  
  
library(targets)  
library(tarchetypes)  
library(here)  
library(rmarkdown)  
library(knitr)  
library(kableExtra)  
library(dplyr)  
library(openxlsx)  
library(ggplot2)  
  
#####  
#  
# '  
# ' Add other packages that will be used in the project below  
# '  
#  
#####
```

11.3. Create placeholder directories

Create placeholder directories for different components of the workflow. These placeholder directories will provide an organising structure to the project and remind you of where to save/store specific files and outputs.

11.4. Create the target script file

Tip

Following are steps on how to create placeholder directories:

- In the lower right pane of RStudio (the file manager pane), find the menu button labelled **Folder**.
- Give this new folder the label of **R**. This folder will hold all bespoke functions that we will create to use for this project workflow;
- Repeat these steps to create new folders with the following labels:
 - **data** - This folder will hold any data that we retrieve as part of this workflow.
 - **outputs** - This folder will hold all our workflow outputs such as plots/figures, tables (in Excel or CSV files), HTML and/or Word and/or PDF outputs
 - **reports** - This folder will hold all our RMarkdown report (.Rmd) files
 - **docs** - This folder will hold any of our deployed outputs such as HTML report, dashboard, etc.

These are placeholder directories which we will populate as we work through the workflow for this project.

11.4. Create the target script file

The next task is to create a `{targets}` script file (`_targets.R`) which is the file that will define the workflow that we will be creating.

11. Creating portable and reproducible scientific workflows

Tip

The `_targets.R` script file can be created through these steps:

- Clicking on **File** -> **New File** -> **R Script** in RStudio.
- A new tab will show in your Source window on the top left quadrant of your RStudio screen. This tab will usually be called `Untitled1`.
- Save this file first and change its name to `_targets.R`. Make sure to save it in the current project directory.
- You know that you were successful in doing this once you see a file called `_targets.R` in the file system window in the lower right quadrant of your RStudio screen.

11.5. Edit the targets script file

Now, the next step is to edit your script file by adding sets of R code that does the following:

- Loads the packages required
- Loads custom functions (if any)
- Defines individual targets using `tar_targets` function
- Ends with a list of targets objects

A basic `{targets}` workflow will look like this:

```
## Load libraries -----  
library(targets)
```

11.5. Edit the targets script file

```
## Load custom functions -----  
for (f in list.files("R", full.names = TRUE)) source (f)  
for (f in list.files(here::here("R"), full.names = TRUE)) source (f)  
  
## Create targets and list targets objects -----
```


12. Contributing to Oxford iHealth projects

This section would be relevant to you if:

1. You have completed the **Open and Reproducible Science in R** sub-module of the MSc for International Health and Tropical Medicine; or,
2. If you are an MSc for International Health and Tropical Medicine alumni and have experience and knowledge using R.

The Oxford iHealth actively runs projects focused on either research software development (based in R) or R-based scientific research workflows. Our list of previous and current projects can be found [here](#).

12.1. Research software development

Oxford iHealth develops research software using the R package system. Our collection of research software tools are found [here](#) and the underlying code is available from our organisational [GitHub](#).

If you would like to contribute to Oxford iHealth's software development, following is a list of steps on how to.

12. Contributing to Oxford iHealth projects

12.1.1. Get familiar with R's package writing process

We build our research software tools using R. Hence, we use the R package writing process. This is described in the official R manual for writing extensions. This manual is the official reference for what is considered acceptable R software development by the R Core Team and are the guides that will ensure that your R package can pass submission and entry into the Comprehensive R Archive Network (CRAN). So, from an academic perspective, you can see this as the official guide of a publisher/publication on what your manuscript should look like so you can submit to their journal for publication. It is not a guarantee of publication, but that you are meeting their publication standards.

However, the R manual for writing extensions is not as easy to navigate and not as easy to read. For beginners, we would recommend starting off with Hadley Wickham's online book called R Packages. The book is free to use online and has clearly delineated chapters and sections for specific R package writing tasks required. We would expect anyone wanting to contribute to the Oxford iHealth's software development projects to have been able to go through this book.

12.1.2. Get familiar and reach intermediate level git and GitHub skills

Our software development process uses git and GitHub to facilitate code sharing and versioning. It is paramount that anyone wanting to contribute to any software project should have at least intermediate level git and GitHub skills. These include:

- Competent in cloning and/or forking software project repositories;
- Competent in the branching process of git and GitHub;
- Competent in the pull request process of GitHub;

12.1. Research software development

- Competent in GitHub's issues tracker and project tracker system;
- Competent in code review process of GitHub.

If you want to brush up on your git and GitHub, please go through Jenny Bryan's *Happy Git and GitHub for the useR*.

12.1.3. Review our portfolio of research software

Visit the packages page of the Oxford iHealth website - <https://oxford-ihtm.io/packages/> to see our current line-up of Oxford iHealth software projects.

Have a look at our R Universe of research software - <https://oxfordihtm.r-universe.dev> - for the build status of each project.

Have a look at our GitHub organisation - <https://github.com/OxfordIHM> - to see the code for each of these projects.

Each project is in continuous development. We recommend looking at the repository for each of the projects, understand via the README what the project is trying to achieve and then review each projects issues page to see what the current line up of tasks or issues that the development requires. If for some reason there is no issues listed in a project, this is most likely that current developers have not gotten around to documenting their tasks/issues at hand. If so, you can make an issue to ask developers what the best task is to do for a beginner to contribute to.

12.1.4. Communicate with developers

Once you have found a project and an issue that you want to work on, make a comment on that issue making sure to tag the developer/maintainer stating that you are going to have a go at this issue and then will make a pull request of your contribution.

12. Contributing to Oxford iHealth projects

This communication is important as this will trigger the developer/maintainer to confirm that you are an eligible member of the Oxford iHealth (the pre-requisites above) and then will add you as a collaborator on the project. This step is important because your status in the project as a collaborator will determine your next step on contributing. We prefer that Oxford iHealth members are internal collaborators as this simplifies their participation (see next section).

12.1.5. Clone or fork the project repository

Once you have been added to the project repository as a collaborator, you can now **clone** the repository to your local machine and then start making your contribution.

If for some reason you haven't been added to the project repository as a collaborator yet but you are itching to contribute, you can still start contributing but instead of cloning, you will need to **fork** the repository. This is similar to cloning but your **fork** is identified as being from someone outside the organisation. This means you will need to make your own GitHub repository under your own username of the project as a fork of the original, write your code contribution and commit to your repository and then make a pull request to the original repository.

These forking steps are described in all the CONTRIBUTING notes/guidance in each project as we open contributions from anyone (not just Oxford iHealth members) but not all are eligible to be internal collaborators.

We would like to avoid this for eligible members of Oxford iHealth so will endeavour to keep up with collaborator requests. So, if you are impatient and would really like to contribute already, consider forking but be warned that this has a lot more complicated steps than simple cloning. Either follow-up your request to be made a collaborator.

References

Jenny Bryan, and Jim Hester. n.d. *Happy Git and GitHub for the useR*. Accessed September 3, 2024. <https://happygitwithr.com/>.

